BaitFL: Lightweight Trap-Based Free-Rider Detection and Mitigation in Federated Learning

Aashnan Rahman
Department of Computer Science and Engineering
Islamic University of Technology
MSc Thesis

***Abstract*—Federated learning relies on participants performing useful local optimization, but free riders can consume the shared model while submitting unchanged or fabricated responses. Detecting such behavior is difficult when honest clients have heterogeneous data and naturally different update statistics. This paper investigates BaitFL, a server-side challenge protocol that compares responses to undisclosed perturbations of the same model. Robust update-norm and tensor-profile measurements first nominate anomalous clients; separate geometric separation tests and repeated fresh evidence authorize removal. Component-specific memory retains strong evidence across ambiguous cycles, while bounded quarantine prevents weak suspicion from causing indefinite exclusion. The method requires no auxiliary detector training, client-reported loss, or attack-specific history reconstruction. We evaluate BaitFL across multiple random seeds on MNIST and CIFAR-10, covering four fabrication strategies, IID and Dirichlet non-IID data, and 30% and 40% free riders over 100 training rounds. The method achieves 100.00% recall and pooled precision of 99.91%. Most executions complete removal at round 30, while difficult cases require up to 90 rounds. There are 3 permanent honest-client removals, and 32 honest client-cases experience temporary quarantine. The findings support the approach under the evaluated settings. Adaptive adversaries and secure aggregation remain outside the study's scope.**

***Index Terms*—federated learning, free-rider detection, hidden challenges, robust statistics, non-IID data, client auditing**

I. Introduction

Federated learning (FL) allows multiple clients to train a shared model without handing over their local data. In each communication round, the server distributes the current global model, the clients update it using their own data, and the server aggregates the returned updates into a revised model [1]. Since raw data stays with its owner, FL is well suited to domains such as healthcare and personal mobile services, where pooling sensitive records in one place is often impractical or undesirable [1], [2]. This protection is not absolute, as shared updates can still leak information [3]. Exchanging updates instead of datasets also avoids moving large volumes of local data, although repeated model exchange remains a significant communication cost in FL [1]. Effective collaboration depends on clients performing useful local training, even though the server cannot directly observe that work.

Like other distributed systems, FL is open to misconduct. Label-flipping and model-poisoning attacks, for example, alter local data or submitted updates in order to degrade the global model or introduce targeted behavior [3], [4]. Free riding is a quieter problem. The participant's primary aim is to obtain the shared model without paying the cost of local training [5], [6]. A client may lack useful data or computational resources, or may simply choose to conserve them. Instead of training, it can return an unchanged model, fabricate a noisy response, or build an update from models received in earlier rounds. Such responses may resemble what the server expects, so they need not visibly disrupt learning and can evade checks aimed at overtly harmful updates. Free riding shifts the training burden to honest participants. When a client withholds useful data and computation, the federation also loses a contribution that could have improved the shared model.

Detecting this omission is difficult because unusual updates are not limited to free riders. Honest clients differ in sample sizes, class distributions, and local learning trajectories, particularly under non-IID data [3]. A small update can therefore reflect legitimate training, while a fabricated one can look like an ordinary contribution. The server has to tell these cases apart without observing local optimization. Existing approaches rely on learned anomaly models, parameter audits, or evidence obtained through inference attacks [5], [7], [8]. This raises a practical question: can lightweight server-side checks detect free riding without training an auxiliary detector? There is also a cost to being wrong. Withholding an honest client's updates can reduce its contribution to the shared model, even if it is never permanently removed. A detector must therefore consider both the evidence needed for removal and the duration of temporary exclusion.

To address this problem, we propose *BaitFL*, a lightweight trap-based method for detecting and mitigating free riders in federated learning.

The main contributions of this work are as follows:

- **Lightweight server-side detection.** We develop a challenge-based detector using update norms and tensor profiles, without training an auxiliary detection model or accessing clients' private training data.
- **Confirmation using repeated evidence.** We design a decision mechanism that separates initial suspicion from permanent removal and requires fresh evidence of the same component across two challenges and distinct audit cycles for statistical confirmation.
- **Bounded mitigation and rehabilitation.** We retain strong evidence across ambiguous cycles while allowing

unconfirmed quarantine to expire and clearing statistical memory after repeated clean observations.

- **Evaluation across four free-rider behaviors.** We study unchanged-model responses, random fabrication, history averaging, and trajectory imitation on MNIST and CIFAR-10 under IID and non-IID partitions, with 30% and 40% free riders.

The remainder of the paper reviews related work, defines the threat model and the free-rider behaviors evaluated, describes the auditing method in detail, and reports the resulting detection performance, computational cost, and limitations.

II. Literature Review

A. Free-Rider Attacks and Evasion

Lin *et al.* studied clients that fabricate updates without local training data, including random submissions and responses derived from changes in the global model [5]. They showed that taking part in model exchange does not establish taking part in training. The evaluated fabrications were detectable, but this does not guarantee that every non-training client will remain distinguishable from honest clients.

Fraboni *et al.* analyzed free riding under iterative model aggregation and developed stochastic disguises that follow the trajectory of the shared model [6]. Their analysis showed that a free rider can approach the model obtained by contributing participants, which makes simple unchanged-model checks insufficient. Zhu *et al.* analyzed local and global training dynamics with an Ornstein–Uhlenbeck model and proposed scaled-delta and advanced free-rider attacks that address both convergence and stealth [9]. These attacks expose a weakness of passive update screening, since a fabricated update can be designed to match the statistics a detector observes. This motivates auditing responses to controlled challenges instead of relying only on how ordinary updates look.

B. Learned Detection and Contribution Auditing

To detect fabricated updates, Lin *et al.* proposed STD-DAGMM, which combines an autoencoder-based representation, reconstruction information, and update standard deviation with mixture-based anomaly scoring [5]. Adding the standard deviation improved separation for attacks that the original DAGMM detector struggled with. This gives richer evidence than a single scalar statistic, but it requires fitting an auxiliary encoder, decoder, and estimation network on high-dimensional updates. Training and storing this additional model adds computation and memory cost that compact update summaries avoid. Section V-G reports the fitting cost measured in our STD-DAGMM implementation.

Wang *et al.* proposed FRAD for AIoT, which combines DAGMM with contribution and reputation information [10]. Contribution is modeled from computing resources, communication cost, and data quality, and a PageRank-based reputation guides participant selection. This extends anomaly detection with historical evidence, and the authors report detection under high free-rider proportions. FRAD still fits an auxiliary detector, however, and it needs resource and quality signals beyond the returned model updates, which are an extra deployment requirement to obtain reliably. Contribution ranking alone also cannot tell whether a low-ranked client skipped training.

Separately, Wang *et al.* introduced PASS, in which clients audit other participants' updates on their own private datasets and return accuracy-based contribution scores [7]. PASS addresses both anonymous and selfish free riders, including evaluated settings where attackers are the majority. Peer auditing, however, adds local model evaluation, update distribution, and score reporting. It also gives clients access to individual updates, a privacy exposure that PASS mitigates with Gaussian perturbation and parameter pruning, at the price of reduced audit utility. The authors leave network-cost validation open, since their standalone simulator does not measure communication. In addition, an accuracy-based score depends on the evaluator's data, so under heterogeneity low utility and absence of training need not coincide.

C. Canary-Based Evidence and Privacy Tradeoffs

Recasens *et al.* proposed FRIDA, which repurposes membership and property inference to obtain evidence that a client trained [8]. Its membership-based variants ask clients to train on server-provided canary examples and assess the effect through loss or gradient similarity. This creates an explicit training signal rather than relying on incidental update statistics. Canary construction and distribution, though, require additional labeled examples, canary training adds client work beyond the original local task, and loss or gradient testing adds server computation. The signal also shows that the canary task was used, not that the client's private data contributed useful work.

FRIDA's property-based variants infer characteristics such as local label distributions and test their consistency or diversity [8]. They avoid canary training but deliberately extract information about local data, which may itself be sensitive. The two families therefore carry different costs: extra training on challenge data for membership tests, and disclosure of inferred dataset properties for property tests.

Secure aggregation, developed by Bonawitz *et al.*, combines client updates without revealing them individually [11]. This protects individual contributions but conflicts with detectors that need attributable responses. BaitFL shares this limitation. Avoiding raw-data access and property inference does not make individual model responses private, and the method cannot be inserted unchanged into standard secure aggregation.

Taken together, these limitations point to three practical requirements: less auxiliary detector training, no additional client-side evaluation or canary tasks, and a clear separation between suspicion and sufficient evidence for removal. BaitFL studies model perturbations and repeated evidence from update norms and tensor profiles under these requirements. Its challenge rounds still consume client work and withhold responses from aggregation, so execution cost and honest-client exclusion must be evaluated alongside detection quality.

III. Threat Model

A. Participants and Server Knowledge

We consider a trusted server coordinating N clients with persistent identities and reliable communication. All active clients respond in every round, with no client sampling, late arrivals, or dropouts. Removed clients no longer participate. Quarantined clients continue to respond. Their responses and all responses to traps are excluded from aggregation. Honest clients train the model they receive using the prescribed optimizer. Their local data may differ in size and class distribution, and the experiments cover both IID and non-IID partitions.

The server can send different model states to different clients. Clients are not told whether the model they receive is an ordinary global model or a trap. For client i in round t, the server knows the model it sent, $w_{t,i}^{\text{send}}$, and the model returned by the client, $w_{t,i}^{\text{ret}}$. It computes the update and its norm as

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \qquad r_{t,i} = \|\Delta_{t,i}\|_2. \tag{1}$$

Screening uses the returned models and the server's audit history. It does not require private training records or prior knowledge of which clients are free riders or how they fabricate their responses. The server observes individual client responses, so this setting does not use secure aggregation.

B. Free-Rider Objective and Scope

The free riders considered here seek access to the shared model without performing local training. A client may have no suitable data, lack the resources needed to train the model, or possess both but choose not to use them. It can still receive and store models and perform the relatively simple calculations needed to fabricate a response. Its objective is to avoid training costs rather than to introduce a targeted error or backdoor into the global model.

We study persistent free riding: a free rider performs no local optimization at any stage of an experiment. Clients that train for fewer local steps or alternate between honest training and free riding are outside this scope. Each free rider uses only its own received-model history and local randomness. Free riders do not collude, share responses, or change identities. Their response strategies do not adapt to identify or evade audits. Adaptive strategies and the possibility of recognizing traps are discussed in Section VIII-C.

We assume an honest majority because the detector uses robust peer statistics as its reference and seeks a separated minority of suspicious responses. The experiments use 30% and 40% free riders. Each experiment evaluates one fixed fabrication strategy for all free riders. The four strategies are defined below.

C. Evaluated Free-Rider Behaviors

For a free rider, the returned model is $w_{t,i}^{\text{ret}} = \widehat{w}_{t,i}$. The following equations apply to floating entries of the model state, including floating buffers. Integer entries are copied unchanged from the received model.

FR1: current-model sendback. The client returns the received model unchanged,

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}}. \tag{2}$$

This produces a zero update without local training, regardless of whether the received model is a trap.

FR2: bounded random fabrication. The client adds independent uniform noise to each floating point entry of the received model state,

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \epsilon_{t,i}, \quad (\epsilon_{t,i})_j \sim \mathcal{U}(-10^{-3}, 10^{-3}). \tag{3}$$

The added noise makes the response differ from the received model without performing any local training.

FR3: received-history averaging. Let H be the history length and $\mathcal{H}_{t,i}$ contain the most recent $\min(H, t)$ model states received by client i. The client returns their mean,

$$\widehat{w}_{t,i} = \frac{1}{|\mathcal{H}_{t,i}|} \sum_{w \in \mathcal{H}_{t,i}} w. \tag{4}$$

The history includes the current model and any traps received during earlier rounds. Averaging these states produces a response based on the shared model's recent evolution without using local training data.

FR4: received-trajectory imitation. The client computes differences between consecutive received models,

$$d_{s,i} = w_{s,i}^{\text{send}} - w_{s-1,i}^{\text{send}}. \tag{5}$$

Let $\overline{d}_{t,i}$ be the mean of up to five most recent differences. The response is

$$\widehat{w}_{t,i} = w_{t,i}^{\text{send}} + \overline{d}_{t,i} + \eta_{t,i}. \tag{6}$$

Once differences are available, the client draws a standard Gaussian vector $z_{t,i}$ and rescales it to obtain

$$\eta_{t,i} = 0.1 \, \frac{\max(\|\overline{d}_{t,i}\|_2, \epsilon_0)}{\max(\|z_{t,i}\|_2, \epsilon_0)} \, z_{t,i}, \qquad \epsilon_0 = 10^{-6}. \tag{7}$$

These vectors contain all floating model entries. For its first response, $\overline{d}_{1,i} = 0$ and $\eta_{1,i}$ instead has independent Gaussian coordinates with standard deviation 10^{-4}. Received traps enter the difference history just like ordinary models, so they also affect the estimated trajectory. This strategy follows recent model changes while adding variation to the fabricated response.

IV. Proposed Methodology

BaitFL audits clients by observing how they respond to perturbed models. The procedure has four phases: warmup, initial paired coverage, confirmation and review, and sparse auditing. Update statistics identify clients for further checks. Permanent removal requires repeated evidence, while temporary exclusion has a separate review budget. The client records $\mathcal{S}$ store each client's status, evidence, and review progress.

A. Challenges, Measurements, and Aggregation

At the start of a paired audit cycle, the server takes a copy $w^{(c)}$ of the current global model and creates two traps, A and B. For floating tensor k, trap $a \in \{A, B\}$ is

$$\tau_{a,k} = w_k^{(c)} + \gamma \max\{\mathrm{sd}(w_k^{(c)}), \epsilon_\tau\}\xi_{a,k}, \quad \xi_{a,k} \sim \mathcal{N}(0, I). \tag{8}$$

The two traps use independent noise. We set $\gamma = 0.01$ and $\epsilon_\tau = 10^{-4}$. Perturbations apply to floating model tensors, including BatchNorm buffers. Integer counters are unchanged. Both traps remain fixed for the cycle, even as ordinary updates change the global model.

For each response, the server records the update norm r_i and a tensor profile p_i. Its coordinate for tensor k is

$$p_{i,k} = \frac{\|\Delta_{i,k}\|_2}{\sum_j \|\Delta_{i,j}\|_2}. \tag{9}$$

When the denominator is below 10^{-6}, the profile is set to zero. The norm measures update magnitude, while the profile describes how that magnitude is distributed across model tensors.

Clients are compared only with others that received the same trap. The reference for client i consists of all other active clients with a response to that trap. Let m_i be the median of their update norms and d_i the median absolute deviation. The normalized norm score is

$$z_i^n = \frac{r_i - m_i}{d_i/\Phi^{-1}(0.75)}, \tag{10}$$

where Φ is the standard normal cumulative distribution function. For the profile score, the server first forms the coordinatewise median of the peer profiles. It measures each profile's Euclidean distance from that reference and normalizes the client's distance using the median and median absolute deviation of the peer distances. This gives z_i^p. A zero median absolute deviation leaves the corresponding score undefined.

Only ordinary responses from eligible clients enter aggregation. Let $\mathcal{A}_t$ be the clients that were neither trapped nor quarantined during round t, after excluding any clients removed or newly quarantined at its end. The aggregation routine used in the algorithms is

$$w_{t+1} = w_t + \frac{1}{|\mathcal{A}_t|} \sum_{i \in \mathcal{A}_t} \Delta_{t,i}. \tag{11}$$

If $\mathcal{A}_t$ is empty, the model is unchanged. Clients receive equal weight. Training on a trap replaces ordinary local training for that round.

B. Phase I: Warmup

The first ten rounds use the ordinary global model. Clients return their responses and the server aggregates them, as shown in Algorithm 1, lines 3 to 6. This gives the federation an initial training trajectory before audits begin. Client evidence counters start at zero, and no client is removed during warmup.

Algorithm 1 Phase I: ordinary training during warmup

```
1: procedure WARMUP(w, S)
2:     for t = 1, ..., 10 do
3:         Send w to all active clients
4:         Collect responses and record their update summaries
5:         A_t ← all active clients
6:         w ← AGGREGATE(w, A_t)
7:     end for
8:     return (w, 10, S)
9: end procedure
```

C. Phase II: Initial Paired Coverage

Rounds 11 to 20 give every client one response to each trap. The server shuffles clients into at most ten groups. It assigns pairs of groups to traps during the first half of the cycle, then repeats the assignments with the opposite trap. With 100 clients, approximately 20% receive a trap in each round. The remaining clients receive the current global model.

Algorithm 2 describes this collection procedure and its reuse in later confirmation cycles. The server chooses one response position for each client and trap before observing the responses, as shown on line 11. It waits for the complete pair before making a cycle decision on lines 21 to 22. The last round's aggregation follows that decision so newly excluded responses do not enter the global model.

1) Screening: The server calculates $D_i = z_i^p - z_i^n$ for trap A, trap B, and their average. For each view, it builds a histogram with $\lceil\sqrt{n_v}\rceil$ equally spaced bins, where n_v is the number of clients with valid paired scores. An Otsu split provides a boundary between the two score groups [12]. The split is accepted only when the lower side of the averaged view contains a majority of active clients. The server then uses 1,000 bootstrap samples to estimate a 99% interval for each boundary [13]. The same resampled clients are used in all three views.

A client above the upper boundary in every view is nominated as suspicious. One below the lower boundary in every view is clear. The remaining clients are candidates for further checking. If a usable split cannot be obtained, norms below the peer median on both traps nominate suspicion. Neither norm below the median gives a clear result. A single low norm or an undefined score gives a candidate result.

2) Geometric evidence: Suspicion must pass an additional separation test before it counts toward removal. On each trap, let $\mathcal{C}$ be the nominated cohort and $\mathcal{P}$ the remaining clients. With n active clients, the cohort must satisfy

$$\left\lceil \frac{n}{\lceil\sqrt{n}\rceil} \right\rceil \le |\mathcal{C}| < |\mathcal{P}|. \tag{12}$$

Quartiles are calculated separately for norms and profile distances. Write $Q_q(\mathcal{C})$ for the empirical quantile at level q and $I_C = Q_{.75}(\mathcal{C}) - Q_{.25}(\mathcal{C})$ for its interquartile range. For low update norms, evidence is accepted when

$$\begin{aligned} Q_{.25}(\mathcal{P}) - Q_{.75}(\mathcal{C}) &> I_C, \\ \min(\mathcal{P}) - Q_{.75}(\mathcal{C}) &> I_C. \end{aligned} \tag{13}$$

Algorithm 2 Phase II: collect a complete pair of trap responses

Require: Training horizon T
procedure PAIREDCYCLE($w, t, \mathcal{S}$, initial)
  Advance the cycle number c
  Create τ_A, τ_B from w using Eq. (8)
  if initial **then**
    Shuffle clients into groups and schedule paired coverage
  else
    Separate clients under review from ordinary clients
    Schedule alternating traps for reviewed clients
    Give ordinary clients one response to each trap
  end if
  Preselect one position per client and trap
  Initialize selected observations O
  $L \leftarrow \min(10, T - t)$
  for $s = 1, \ldots, L$ **do**
    $t \leftarrow t + 1$
    Read the scheduled trap and recipient set $\mathcal{T}_t$
    Send the trap to $\mathcal{T}_t$ and w to other clients
    Collect responses and store the preselected observations
    $\mathcal{A}_t \leftarrow$ ordinary clients not quarantined at round start
    if $s = 10$ **then**
      $E \leftarrow$ SCREENANDCERTIFY(O)
      $\mathcal{S} \leftarrow$ CONFIRM($\mathcal{S}, E, c$)
      Exclude new removals and quarantines from $\mathcal{A}_t$
    end if
    $w \leftarrow$ AGGREGATE($w, \mathcal{A}_t$)
  end for
  Update the audit clock from the remaining review state
  return $(w, t, \mathcal{S})$
end procedure

A nominated client must also have a norm below the median of $\mathcal{P}$. These conditions require separation from the peer population, rather than concentration within the nominated group alone.

For high profile distances, define $I_P = Q_{.75}(\mathcal{P}) - Q_{.25}(\mathcal{P})$ and $g = Q_{.25}(\mathcal{C}) - Q_{.75}(\mathcal{P})$. Evidence requires $g > I_C$, $g > 0$, and either

$$Q_{.25}(\mathcal{C}) - \max(\mathcal{P}) > I_C \quad \text{or} \quad \frac{I_C}{|\mathcal{C}| - 1} < \frac{I_P}{|\mathcal{P}| - 1}. \tag{14}$$

The client's distance must exceed the median of $\mathcal{P}$. A norm or profile certificate is recorded only when the same component supports that client on both traps. Exact copies of the current or immediately preceding sent model are recorded separately on each trap.

After these checks, S denotes strong paired evidence, C denotes an unresolved candidate, and R denotes a clear cycle result. Only a paired geometric certificate contributes to statistical confirmation. The routine SCREENANDCERTIFY in Algorithm 2 applies these rules and returns the component certificates, exact replay flags, and cycle outcomes.

D. Phase III: Confirmation and Bounded Review

The server keeps separate certificate counts $M_{i,n}$ and $M_{i,p}$ for each client. A fresh paired certificate increases its own count once in a completed cycle. Ambiguous cycles retain the counts. Two consecutive clear cycles erase them. Statistical removal requires a fresh certificate whose component count reaches two, as shown in Algorithm 3, lines 7 to 19. Norm evidence in one cycle and profile evidence in another do not confirm one another. Exact replay uses a separate rule that requires evidence on both traps in two consecutive completed cycles. With the ten warmup rounds and ten rounds per paired cycle, the earliest removal is at round 30.

A fresh strong result places the client in quarantine and starts a review budget of two cycles. Another fresh strong result renews that budget. Otherwise, it decreases after each completed cycle. When the budget expires, the client is released even if unresolved evidence remains in memory. A candidate without strong evidence may contribute on ordinary rounds. Two consecutive clear results rehabilitate a suspected or candidate client. One additional clear cycle completes probation before routine auditing resumes. Clients released from unresolved review have a cooldown of two sparse audits before weak evidence can enroll them again. Fresh paired evidence can still start review.

During confirmation, reviewed and ordinary clients are grouped separately. At most ten groups are allocated in proportion to the two populations. Reviewed clients receive alternating traps throughout the cycle. Ordinary clients receive each trap once. As in the initial cycle, the decision uses only the response positions selected in advance. Each new cycle creates fresh traps. Algorithm 2 collects these responses, and Algorithm 3 updates the client records at the cycle boundary.

E. Phase IV: Sparse Audits

When no client remains under review or probation, the server returns to ordinary training and schedules a sparse audit after ten rounds. In an audit round, all active clients receive the same fresh trap, so the global model remains unchanged for that round. Ordinary rounds between audits use Eq. (11).

The audit applies the same norm and profile measurements to one trap. Screening nominates a cohort, and the geometric tests identify clients for further review. Exact replay also triggers review. Algorithm 4, lines 5 to 10, shows this step. Eligible clients enter a two cycle review without immediate quarantine. The server then resumes paired cycles. A sparse audit cannot increase statistical certificate counts, clear certificate memory, or authorize removal.

F. Overall Procedure

Algorithm 5 connects the four phases. The client records $\mathcal{S}$ contain membership, quarantine and review status, certificate counts, clean progress, probation, and cooldowns. After warmup, every client receives initial paired coverage. Further paired cycles run while review or probation is pending. Otherwise, the server uses ordinary training with sparse audits until the training horizon is reached.

Algorithm 3 Phase III: confirm evidence and manage review

procedure CONFIRM($\mathcal{S}, E, c$)
  for all active clients i **do**
    $J_i \leftarrow$ components certified on both traps in E
    Update the consecutive paired exact replay count
    if $J_i \neq \emptyset$ **then**
      for all $j \in J_i$ **do**
        $M_{i,j} \leftarrow M_{i,j} + 1$
      end for
      Reset the consecutive clear cycle count
    else if the cycle outcome is R **then**
      Advance the consecutive clear cycle count
      if two consecutive clear cycles are complete **then**
        $M_{i,n} \leftarrow 0$, $M_{i,p} \leftarrow 0$
      end if
    else
      Reset the clear cycle count and retain certificate memory
    end if
    if a fresh component count reaches two or exact replay is confirmed **then**
      Permanently remove i
    else if $J_i \neq \emptyset$ or paired exact replay is present **then**
      Quarantine i and set its review budget to two cycles
      Reset its clean progress, probation, and cooldown
    else
      Decrease the review budget toward zero
      Apply clear cycle rehabilitation and probation
      if unconfirmed review has just expired **then**
        Release i and set a cooldown of two sparse audits
      end if
    end if
  end for
  return $\mathcal{S}$
end procedure

Algorithm 4 Phase IV: ordinary training and sparse auditing

procedure SPARSEROUND($w, t, \mathcal{S}$)
  $t \leftarrow t + 1$
  if a sparse audit is due **then**
    Create a fresh trap and send it to all active clients
    Collect responses and compute norm and profile scores
    $\mathcal{F} \leftarrow$ clients selected by geometric evidence or exact replay
    Identify clients currently in audit cooldown
    Decrease audit cooldowns toward zero
    for all $i \in \mathcal{F}$ not in the identified cooldown set **do**
      Open two cycles of review without quarantine
      Reset clean progress and consecutive exact replay evidence
    end for
    Update the audit clock from the screening outcome
  else
    Send w to all active clients and collect responses
    $\mathcal{A}_t \leftarrow$ ordinary clients not quarantined
    $w \leftarrow$ AGGREGATE($w, \mathcal{A}_t$)
  end if
  return $(w, t, \mathcal{S})$
end procedure

Algorithm 5 Overall BaitFL procedure

Require: Initial model w, clients, training horizon $T \geq 30$
Initialize client records $\mathcal{S}$ and zero evidence counts
$(w, t, \mathcal{S}) \leftarrow$ WARMUP($w, \mathcal{S}$)
initial $\leftarrow$ true
while $t < T$ and active clients remain **do**
  if initial or review or probation is pending **then**
    $(w, t, \mathcal{S}) \leftarrow$ PAIREDCYCLE($w, t, \mathcal{S}$, initial)
    initial $\leftarrow$ false
  else
    $(w, t, \mathcal{S}) \leftarrow$ SPARSEROUND($w, t, \mathcal{S}$)
  end if
end while
return w and the final client records $\mathcal{S}$

V. Results

A. Experimental Setup

We evaluate BaitFL on MNIST and CIFAR-10, which contain handwritten digit images and color images from ten object classes, respectively [14], [15]. The evaluation covers four free-rider behaviors, IID and non-IID client data, and free-rider proportions of 30% and 40%. Each configuration is repeated using multiple random seeds, and Table I reports the corresponding mean performance. The optimizer and detector settings remain fixed across replications.

Each federation contains 100 clients and trains for 100 communication rounds. For IID experiments, data are randomly divided into client partitions of equal size. For non-IID experiments, class proportions are allocated using a Dirichlet distribution with $\alpha = 0.5$, with allocation repeated if a client receives no samples. Replications vary data partitioning, attacker assignment, model initialization, training randomness, and audit scheduling.

Honest clients perform three local SGD epochs per round with learning rate 0.01, momentum 0.9, and batch size 32. FR3 uses history length $H = 5$. The MNIST model has two convolutional blocks with 16 and 32 channels and a 128-unit hidden classifier. The CIFAR-10 model has three convolutional blocks with BatchNorm and 32, 64, and 128 channels, followed by a 256-unit hidden classifier. Images are normalized without data augmentation, and all experiments use CPU execution.

B. Evaluation Metrics

A positive detection is *permanent removal*, not nomination or quarantine. For true positives TP, honest removals FP, and missed free riders FN,

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}. \tag{15}$$

Precision is undefined if no client is removed. Complete-detection round t_{all} exists only when every free rider has been removed; otherwise it is NR. Final global test accuracy is a separate utility metric. We additionally report final honest-candidate counts, honest clients ever quarantined, quarantine client-rounds, trap client-rounds, and all excluded active honest client-rounds. These last categories can overlap and must not be summed as disjoint costs.

C. Permanent Detection and Model Utility

Table I groups the results by dataset, partition, and free-rider proportion. Precision and recall are calculated over the four attack types within each seed and then averaged across seeds. Final test accuracy is averaged separately for FR1 to FR4, while round time averages all attack types and seeds with equal weight.

Every free rider is removed in the evaluated replications. Pooled over all replications, precision is 99.91%, recall is 100.00%, and F_1 is 99.96%. The three honest removals occur in MNIST non-IID at 40% free riders, with two in an FR3 execution and one in an FR4 execution. Complete detection occurs between rounds 30 and 90. Most executions finish at round 30, while some MNIST FR4 and non-IID CIFAR-10 FR3 or FR4 executions require additional audit cycles.

Mean final accuracy ranges from 98.83% to 99.02% on MNIST and from 74.51% to 77.84% on CIFAR-10. The diagnostic figures and detailed exclusion records below examine a representative replication. Its accuracy trajectories appear in Figure 5, with full trajectories and removal curves retained in Appendix B. These trajectories illustrate individual executions rather than averages across seeds.

TABLE I
FREE-RIDER DETECTION PERFORMANCE AND GLOBAL MODEL UTILITY.

Dataset	Partition	FR (%)	Detection (%)		Final test accuracy (%)				Round time (s)
			Precision	Recall	FR1	FR2	FR3	FR4	
MNIST	IID	30	100.00	100.00	99.01	99.01	99.02	98.90	42.38
		40	100.00	100.00	98.93	98.94	98.92	98.83	37.05
	Non-IID	30	100.00	100.00	98.92	98.93	98.93	98.91	41.62
		40	99.39	100.00	98.91	98.91	98.90	98.88	36.08
CIFAR-10	IID	30	100.00	100.00	77.84	77.84	77.76	77.77	99.35
		40	100.00	100.00	77.07	77.13	76.92	76.97	85.35
	Non-IID	30	100.00	100.00	75.94	75.86	75.77	75.78	92.30
		40	100.00	100.00	74.70	74.88	74.67	74.51	77.03

Values are averaged across seeds. Precision and recall pool permanent removals across FR1–FR4 within each seed. Test accuracy is averaged separately for each attack. Round time averages the four attack types and includes local training, aggregation, evaluation, and detection.

D. Representative Score Geometry and Evidence Evolution

In the representative replication, Figures 1 and 2 show every client's round-20 norm/profile position at 40%, covering all four attacks and both partitions. The 30% counterparts are retained in Appendix B. Coordinates are paired averages over the two selected trap responses, transformed by asinh solely for display. This monotone transformation preserves ordering while keeping extreme profile-z outliers from flattening the central cloud. There is no clipping, jitter, label-based reference selection, or fitted decision line. Coincident points, particularly exact sendbacks, can conceal multiplicity. The actual decision continues to test each trap separately; a paired scatter point is a diagnostic, not a substitute for the full certificate.

The more demanding CIFAR-10 non-IID FR3 executions are examined directly in Figure 3. Shared population bins expose raw trap-A norm distributions before and at confirming cycles, while annotations report the *paired* fresh norm-certificate counts from the decision logs. The certificate can include honest clients: a visually compact low-norm population is not equivalent to a pure attacker cluster. At 40%, nine honest clients receive the round-20 paired norm certificate, but none receives it at round 60. This explains why nomination quality and eventual removal quality differ. At 40%, the round-20 and round-60 certificates confirm all free riders; at 30%, the relevant pair is round 40 and round 80. The selected panels explain this recorded mechanism, not a new decision rule selected from the histograms.

Figure 4 supplements the raw distributions with class mean norm-z and profile-z over complete cycles, with interquartile client dispersion. Class labels are used only after scoring to describe observed behavior; the server never calculates an "honest-only" boundary. A lower free-rider mean alone is insufficient: the strict certificate must still establish cohort support and separation from the peer range on both traps. Because reference populations and MAD scales change after removal, z-score trajectories are not fixed-unit measurements of model improvement. Undefined scores remain missing. The first ten warmup rounds do not provide these paired trap scores, and no warmup score trajectories are fabricated.

E. Honest-Client Impact

Across the repeated experiments, 3 honest client-cases are permanently removed and 32 experience temporary quarantine. To examine exclusion in detail, Table II retains representative executions and expands FR3 into individual dataset/partition/share rows rather than pooling it with the three easier attacks. It distinguishes distinct quarantined clients, quarantine duration, deliberate trap exclusion, and additional non-trap exclusion. In this representative replication there are 11 honest client-cases ever quarantined, totalling 220 quarantine client-rounds. Nine occur in CIFAR-10 non-IID FR3 at 40%, one in its 30% counterpart, and one in MNIST non-IID FR3 at 40%. None remains quarantined at completion.

There are 296 final honest-candidate client-cases in this replication. Weak candidate labels persist in several completed runs. The most pronounced cases are 34 of 60 honest clients in MNIST non-IID FR3 at 40%, 52 of 60 in CIFAR-10 non-IID FR3 at 40%, and all 70 remaining clients in CIFAR-10 non-IID FR3 at 30%. These are not counted as permanent false positives, but they are reported because screening and final-removal quality are different properties.

F. Execution Cost

For the representative replication, Table III reports means across the four attacks, giving each attack equal weight, with the attack-to-attack range of both round time and peak RSS. MNIST requires approximately 34.6–36.4 seconds per round at 40% free riders and 41.3–43.0 seconds at 30%. CIFAR-10 requires 79.5–90.7 and 97.1–101.2 seconds, respectively. The measurements include serial client simulation, local training, aggregation, and evaluation. The logged round timer ends before checkpoint/log publication; reported execution duration additionally includes initialization, publication work, and recovery work represented by the resumed timer. Powered-off waiting is not a detector computation cost.

Process RSS includes simulated attacker histories and training tensors, not only server-side detection state. Neither round duration nor peak RSS isolates the defense's incremental overhead. A detector-only profiler and controlled no-detector baseline are required before assigning an overhead percentage.

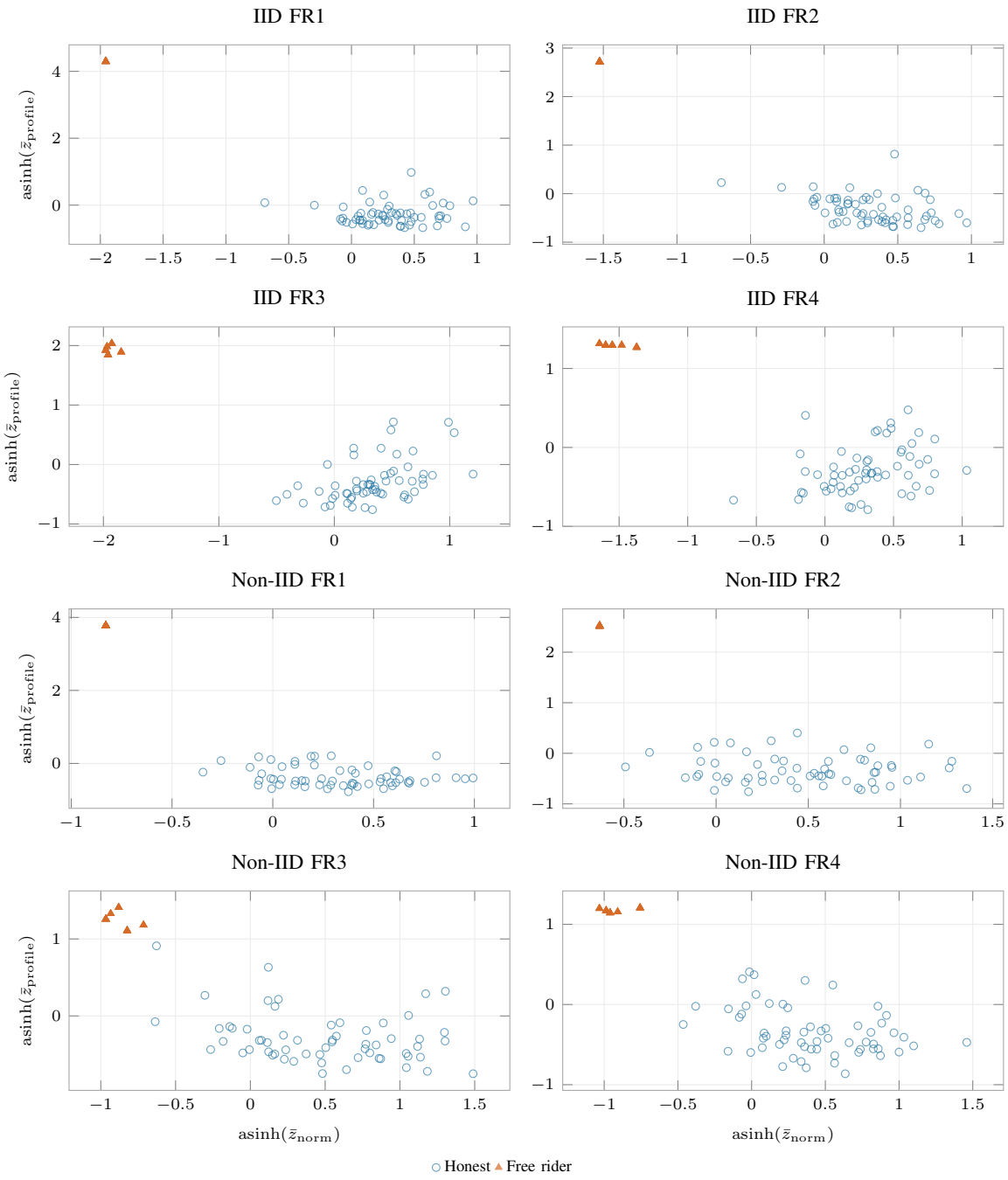


Fig. 1. MNIST: round-20 score geometry at 40% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

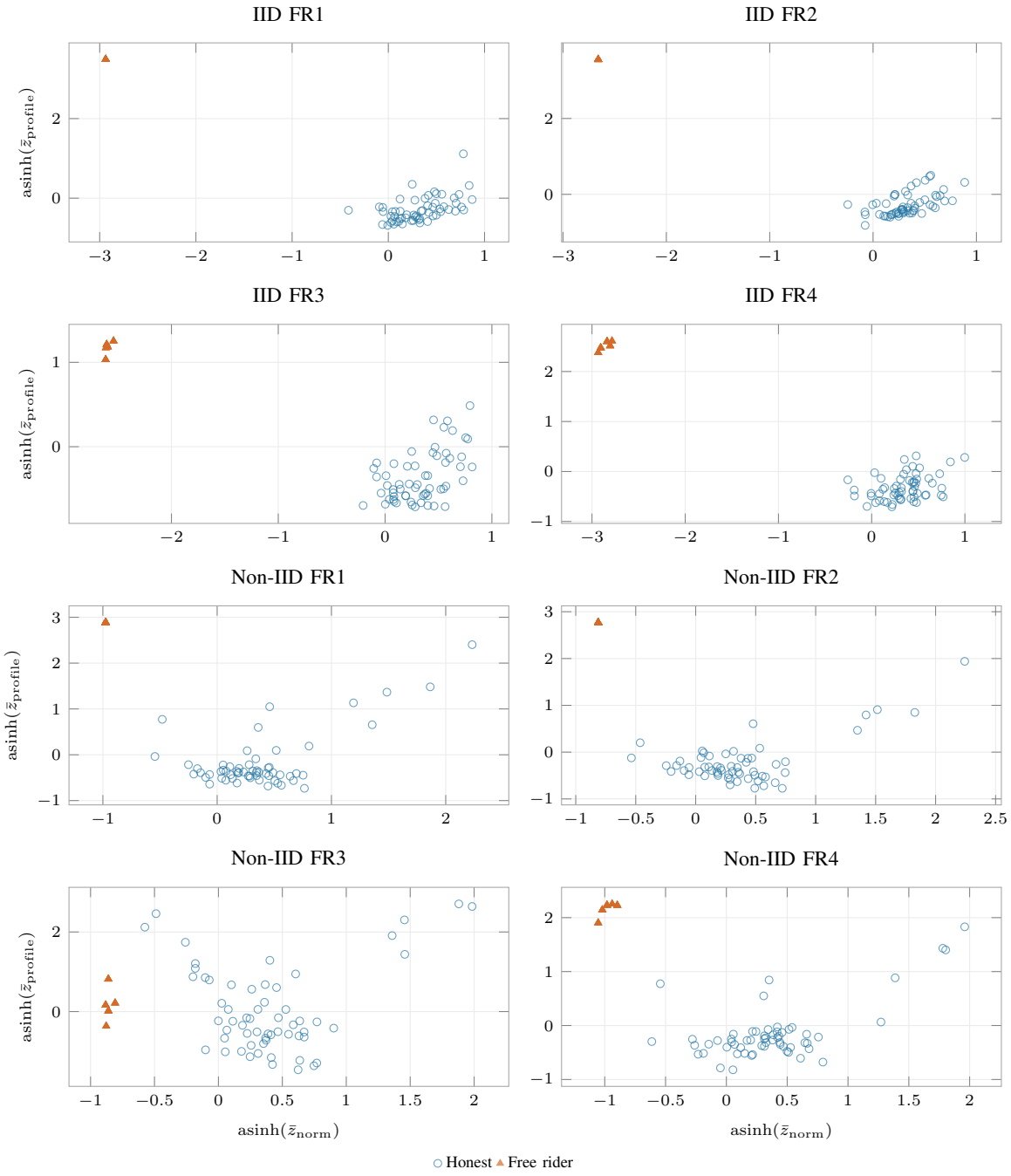


Fig. 2. CIFAR-10: round-20 score geometry at 40% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

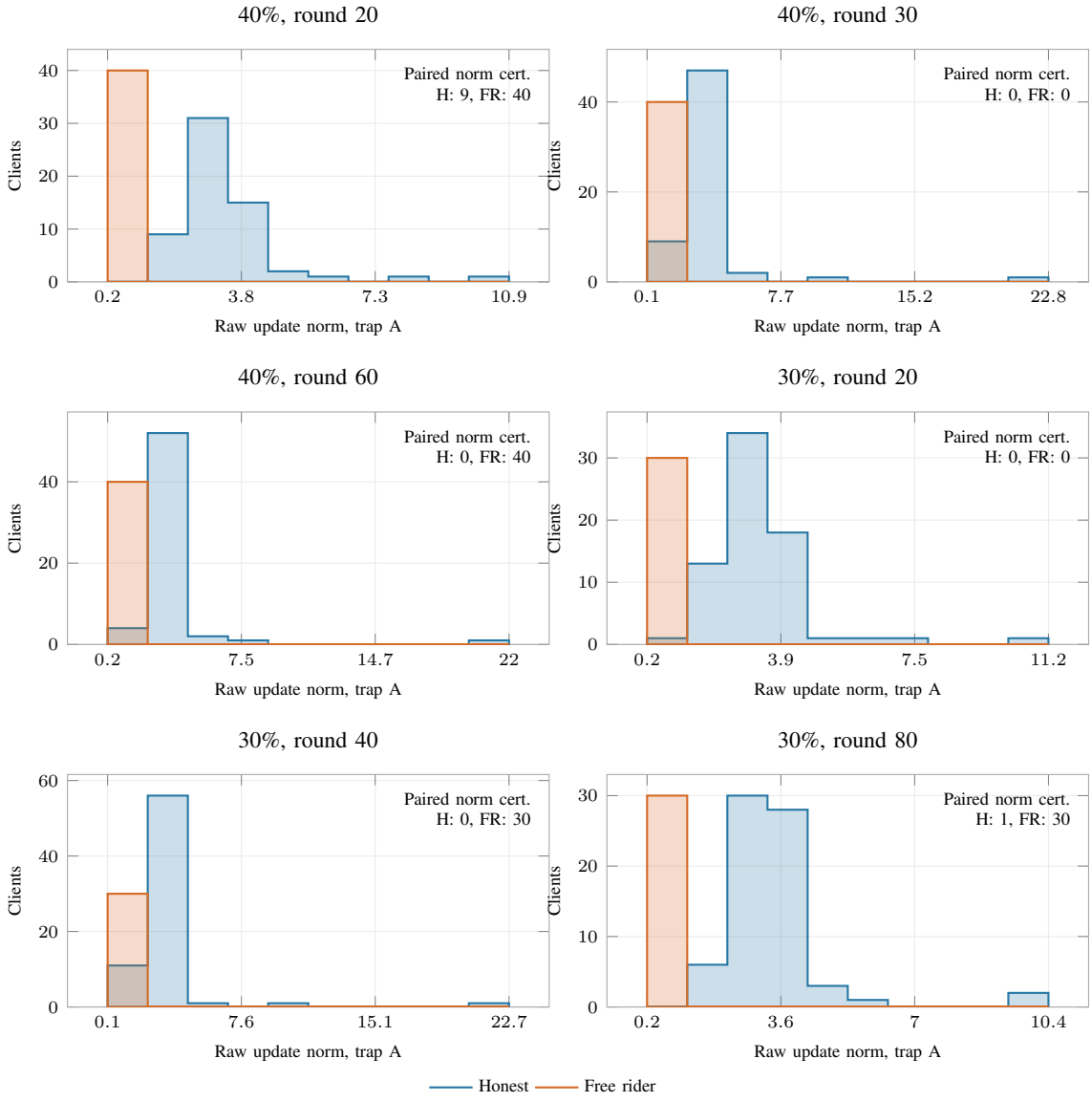


Fig. 3. CIFAR-10 non-IID FR3: raw selected trap-A norm distributions. Shared population-derived equal-width bins within each panel, independent axes across panels; colors may overlap. Annotations count FRESH norm certificates on BOTH traps (H: honest, FR: free rider). Confirming cycles are 20/60 at 40% and 40/80 at 30%; the other panels show ambiguity. A visual gap alone cannot authorize removal.

Figure 6 keeps the per-attack breakdown behind the grouped table: the higher-memory FR3/FR4 executions include simulated attacker histories as well as training and detector tensors. It would be incorrect to attribute that entire difference to server-side detection.

Table IV compares the three completed same-setting v15.3 CIFAR-10 non-IID 40% executions against v15.4. The unfinished FR1 is excluded explicitly, not treated as a completed measurement. This historical comparison is useful context for the concern that evidence memory might be expensive; it is not an isolated overhead experiment. Different detection trajectories, client work, restarts and host load prevent interpreting the differences as a causal speedup. Neither historical nor current logs measure network bytes or detector-only CPU time. Section VII therefore separates measured process resources from calculated algorithmic requirements.

G. Measured 100-Round Native-Workflow Timing

We additionally execute 100 ordinary-update STD-DAGMM rounds per dataset: 100 clients throughout, non-IID, 40% FR3, three local SGD epochs, CPU and six PyTorch threads. This is a paper-guided implementation, not author code. Each round freshly fits the full-update autoencoder/mixture for a declared budget of five epochs and records fitting and scoring separately. This budget is not evidence of detector convergence or an exact replication of an unspecified published training schedule. Flags remain observational: no traps, our evidence gates, or permanent removal are added.

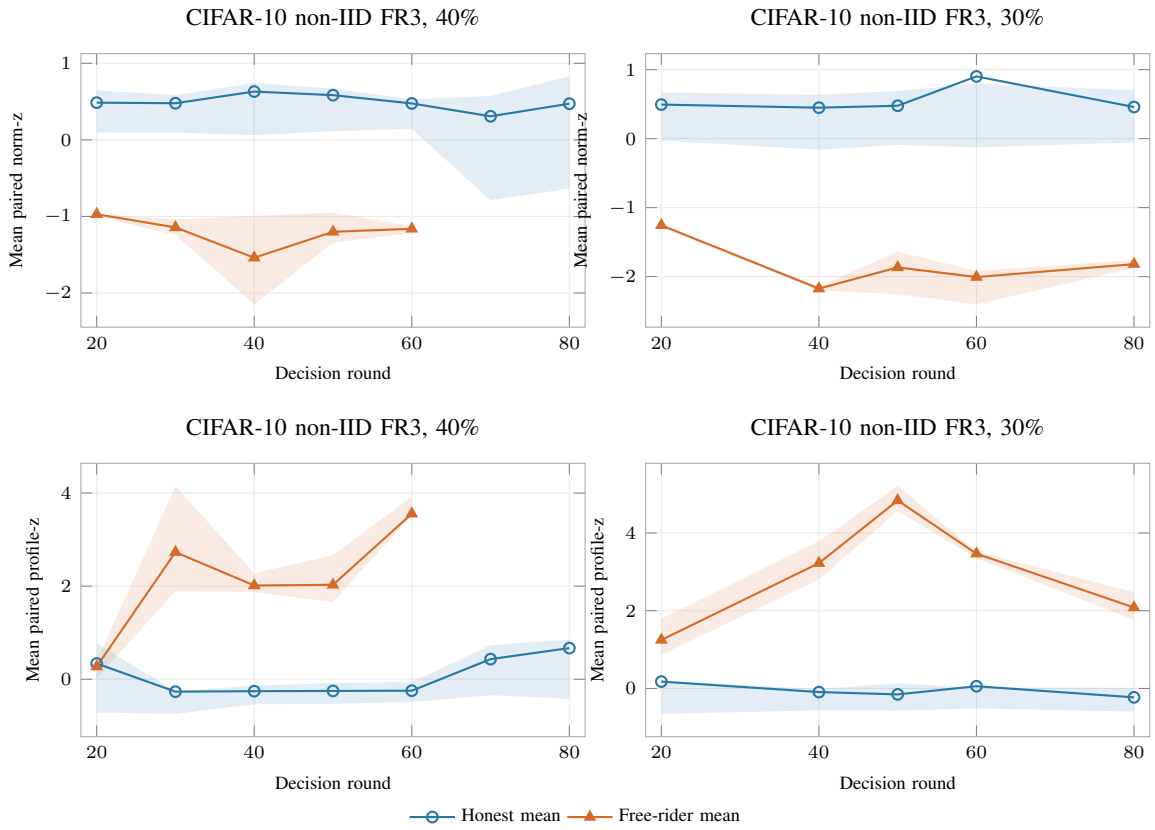


Fig. 4. CIFAR-10 non-IID FR3: mean paired scores over complete evidence cycles through round 80 (lines), with client interquartile intervals shaded. Scores are first averaged over A/B per client. Bands show client dispersion, NOT mean uncertainty. References/MAD scales change each cycle; z-score changes need not match raw-norm changes. Free-rider curves stop at removal, never zero-filled. Class colors are for evaluation only.

TABLE II
HONEST-CLIENT EXCLUSION IN REPRESENTATIVE EXECUTIONS.

Dataset	Partition	FR (%)	Attack	Final honest candidates	Ever quarantined	Quarantine client-rounds	Trap client-rounds	Non-trap excluded	All excluded (%)
MNIST	IID	30	FR3	8/70	0	0	770	0	11.00
MNIST	Non-IID	30	FR3	8/70	0	0	770	0	11.00
MNIST	IID	40	FR3	5/60	0	0	660	0	11.00
MNIST	Non-IID	40	FR3	34/60	1	20	736	1	12.28
CIFAR-10	IID	30	FR3	2/70	0	0	770	0	11.00
CIFAR-10	Non-IID	30	FR3	70/70	1	20	1160	0	16.57
CIFAR-10	IID	40	FR3	4/60	0	0	660	0	11.00
CIFAR-10	Non-IID	40	FR3	52/60	9	180	1104	5	18.48

Candidate entries are unresolved honest clients / honest population. Quarantine client-rounds count clients times rounds, and can overlap trap client-rounds. Non-trap excluded counts only withheld honest responses not already explained by traps; the last column divides all withheld honest responses by active honest client-rounds. The complete per-run appendix retains the other attacks.

Table V and Figure 7 distinguish measured components from missing measurements. Our completed study logged whole-round durations, not separate client-training, aggregation, trap-generation or detection timers; gray totals cannot retrospectively identify those costs. Our feature collection occurs during client responses, whereas complete-cycle scoring and state decisions occur at coverage boundaries; single audits are separate events. Therefore detection is not simply zero on every non-tenth round. Neither local simulator measures real network transmission. STD input serialization and worker startup are harness work, not network communication.

For our actual horizon, the amortized time is $\bar{t}_{100} = \sum_{r=1}^{100} t_r/100$; summing all rounds includes expensive probing/decision rounds and lighter rounds without selecting either subset. Table VII and Figure 8 explain how round frequency affects this average. The figure's left panels show the mean duration of a whole round in each category; its right panels multiply that mean by the category's round count and divide by 100. The five right-hand bars therefore sum to the observed full-run mean. For example, MNIST's three cycle-decision rounds

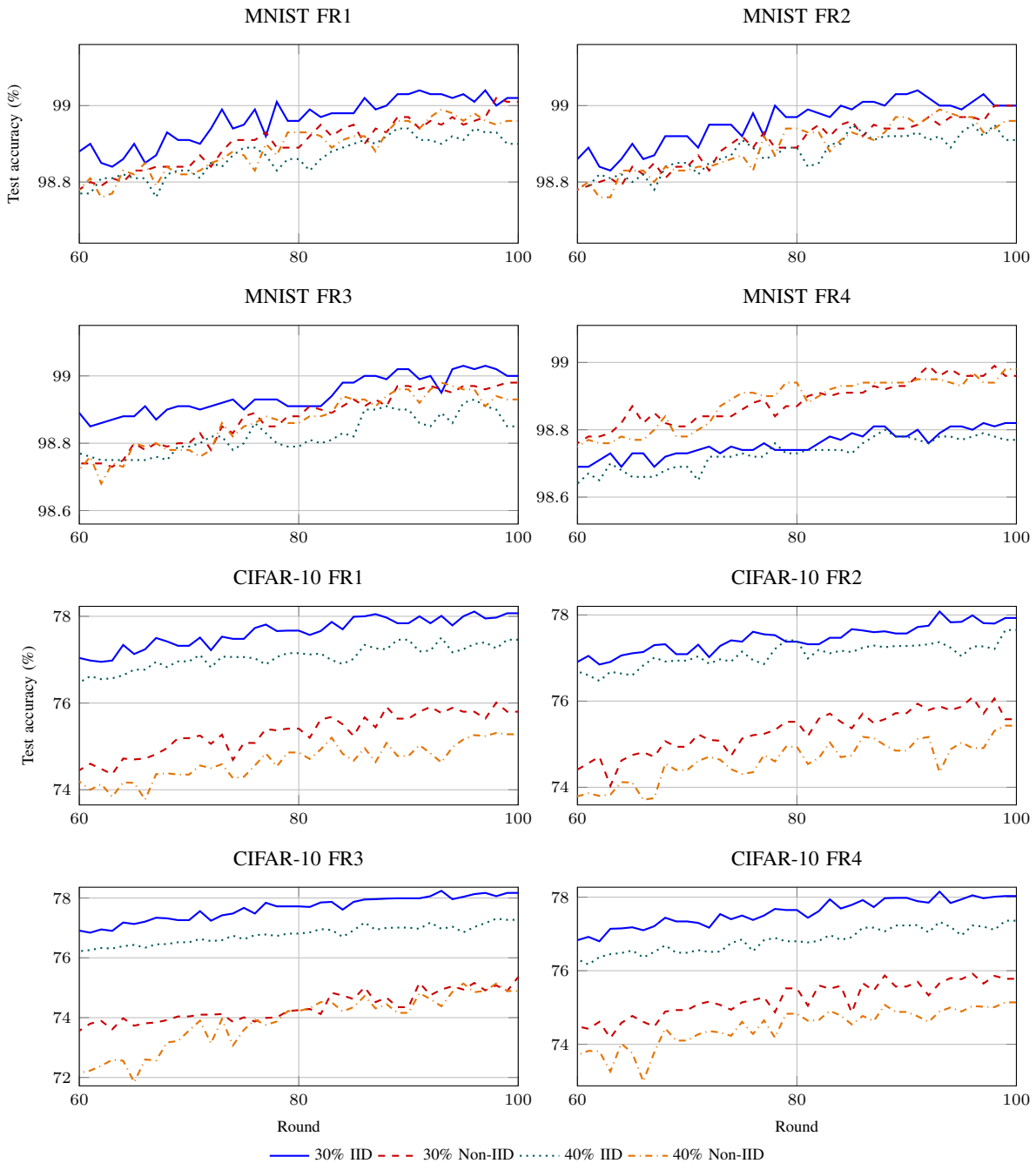


Fig. 5. Late-training global test accuracy (rounds 60–100; the representative replication). Each panel uses its OWN data-derived vertical range in percentage points to expose differences hidden by a 0–100 percent scale. Compare numeric ticks, not apparent slope across panels. Full trajectories appear in the appendix; no smoothing or runs are omitted.

TABLE III
CPU RESOURCES IN REPRESENTATIVE EXECUTIONS: MEAN [MINIMUM–MAXIMUM] ACROSS FR1–FR4. EXECUTION IS MEAN MINUTES PER RUN.

Dataset	Partition	FR (%)	s/round: mean [range]	Peak RSS GiB: mean [range]	min/run
MNIST	IID	30	43.0 [42.3–44.0]	0.53 [0.29–0.72]	75.9
MNIST	Non-IID	30	41.3 [40.7–41.7]	0.56 [0.30–0.75]	73.2
MNIST	IID	40	36.4 [35.3–39.6]	0.59 [0.34–0.85]	65.6
MNIST	Non-IID	40	34.6 [34.3–34.9]	0.61 [0.31–0.85]	62.4
CIFAR-10	IID	30	101.2 [88.2–119.1]	1.28 [0.76–1.75]	178.6
CIFAR-10	Non-IID	30	97.1 [89.6–108.4]	1.30 [0.74–1.76]	170.4
CIFAR-10	IID	40	90.7 [77.4–104.2]	1.38 [0.72–1.93]	165.7
CIFAR-10	Non-IID	40	79.5 [75.8–84.8]	1.50 [0.78–2.13]	143.6

Peak RSS includes local training, aggregation, logged evidence and simulated attacker histories. Round timing ends before checkpoint/log publication. Neither column isolates detector overhead or controls machine load.

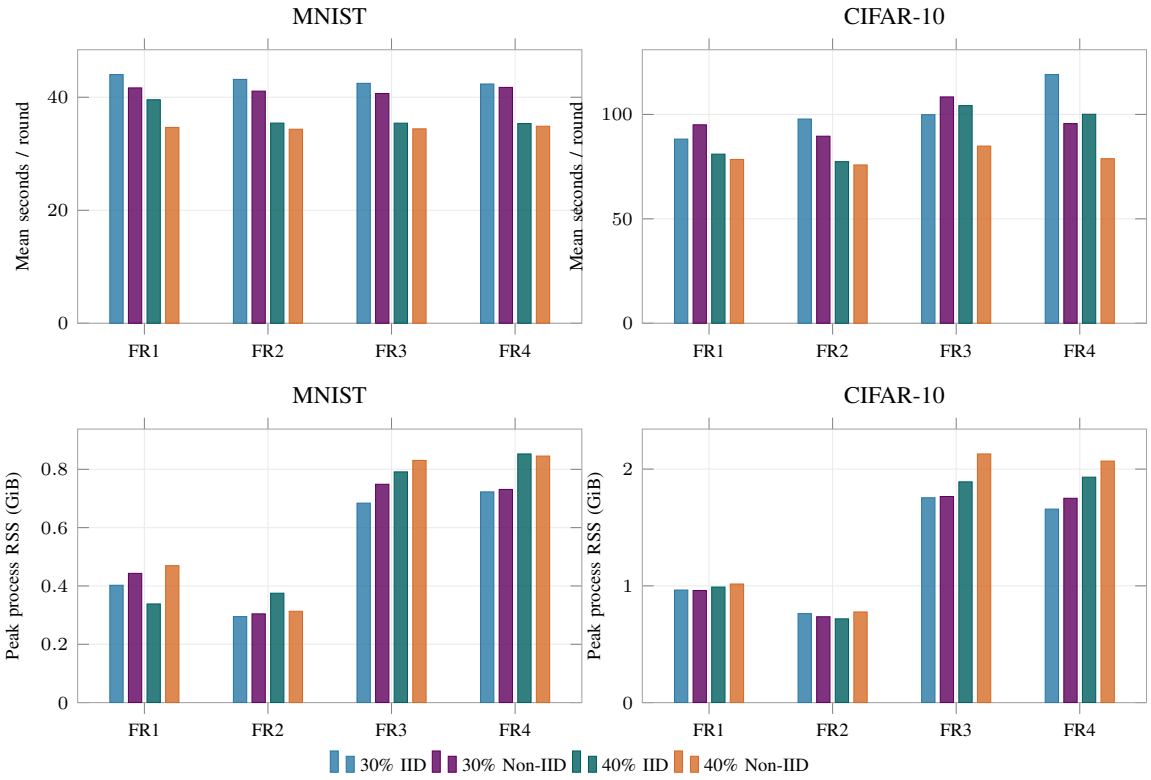


Fig. 6. Attack-specific whole-simulator resource profiles for the representative replication: mean logged round time (top) and peak process RSS (bottom). Four bars per attack distinguish partition and free-rider share. Dataset panels use independent numeric scales. FR3/FR4 memory includes simulated history storage; these bars do not measure detector-only overhead.

average 34.45 seconds each and contribute 1.03 seconds to the 34.41-second full-run average. The 1.03 seconds is not an estimate of detector-only cost. All workflow comparisons use the full 100 executed rounds per method and dataset. Figure 9 shows both complete trajectories.

These are native-workflow costs, not a controlled speedup experiment: our trajectory removes clients and changes probing/aggregation, while the STD execution retains all 100. Historical local attacker evaluation, timer coverage, machine load and data trajectories also differ. CIFAR's detector has a sampled peak worker RSS of about 3.66 GiB and minimum available system memory near zero; paging may confound timing, although paging itself was not measured. This experiment does not establish universal STD-DAGMM detection quality or end-to-end removal effectiveness. Per-round components, phase sums and input hashes are retained with the reporting artifacts. The measured mean round compute is 50.19 seconds on MNIST and 127.24 seconds on CIFAR-10; detector fitting contributes 13.63 and 45.03 seconds, respectively. In these selected non-IID FR3 runs, no free rider is ever flagged, whereas all 60 honest clients are flagged at least once (not all in the same round). These observational flags are not removals and do not validate the chosen five-epoch recipe as an effective detector; neither do they establish a universal limitation of STD-DAGMM. Removing non-training free riders does not itself remove honest SGD jobs: the savings can concern response handling,

TABLE IV

HISTORICAL WHOLE-PROCESS COMPARISON: CIFAR-10, NON-IID, 40%, SEED 42, CPU, 100 ROUNDS. ONLY COMPLETED MATCHING V15.3 RUNS ARE INCLUDED.

	Mean s/round		Peak RSS (GiB)		Execution (min)	
Attack	v15.3	v15.4	v15.3	v15.4	v15.3	v15.4
FR2	78.32	75.80	0.78	0.78	133.06	128.91
FR3	85.32	84.85	1.89	2.13	163.38	160.10
FR4	86.50	78.79	1.81	2.07	169.51	149.52

The unfinished v15.3 FR1 is not reconstructed. Different trajectories, machine load and restarts confound causal overhead claims. These are not isolated detector benchmarks.

simulated attacker diagnostics, aggregation and detection, rather than a proportional reduction of genuine local-training work. The observations cannot isolate those causes.

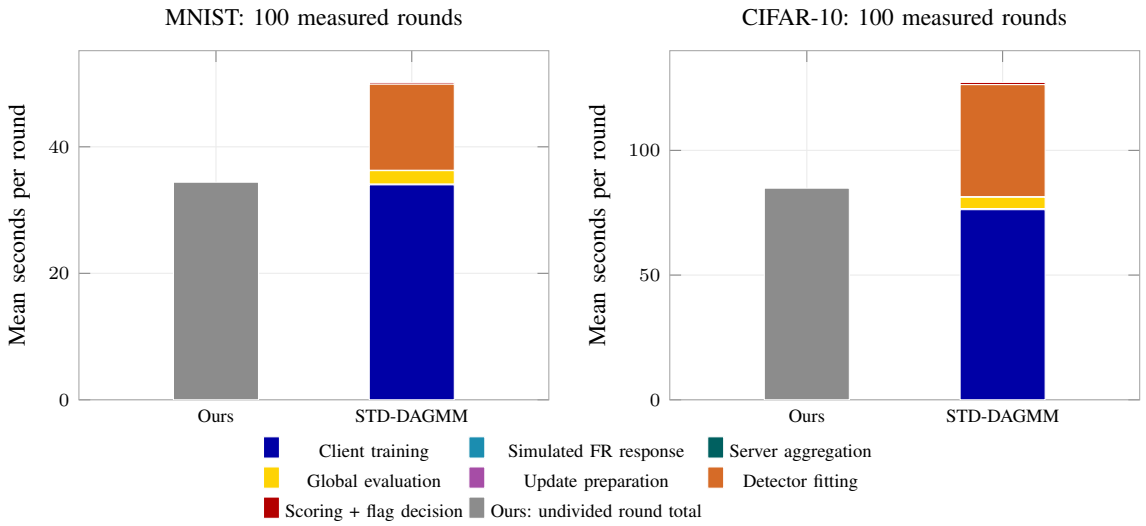


Fig. 7. Mean timing over 100 executed rounds per method and dataset: non-IID FR3, seed 42, 100 initial clients, 40% free riders. STD-DAGMM has measured task components; our gray bars are undivided whole-round totals, not zero-cost detection. Both means include rounds 1–100. Different trajectories, timer coverage and host load prevent a causal speedup claim.

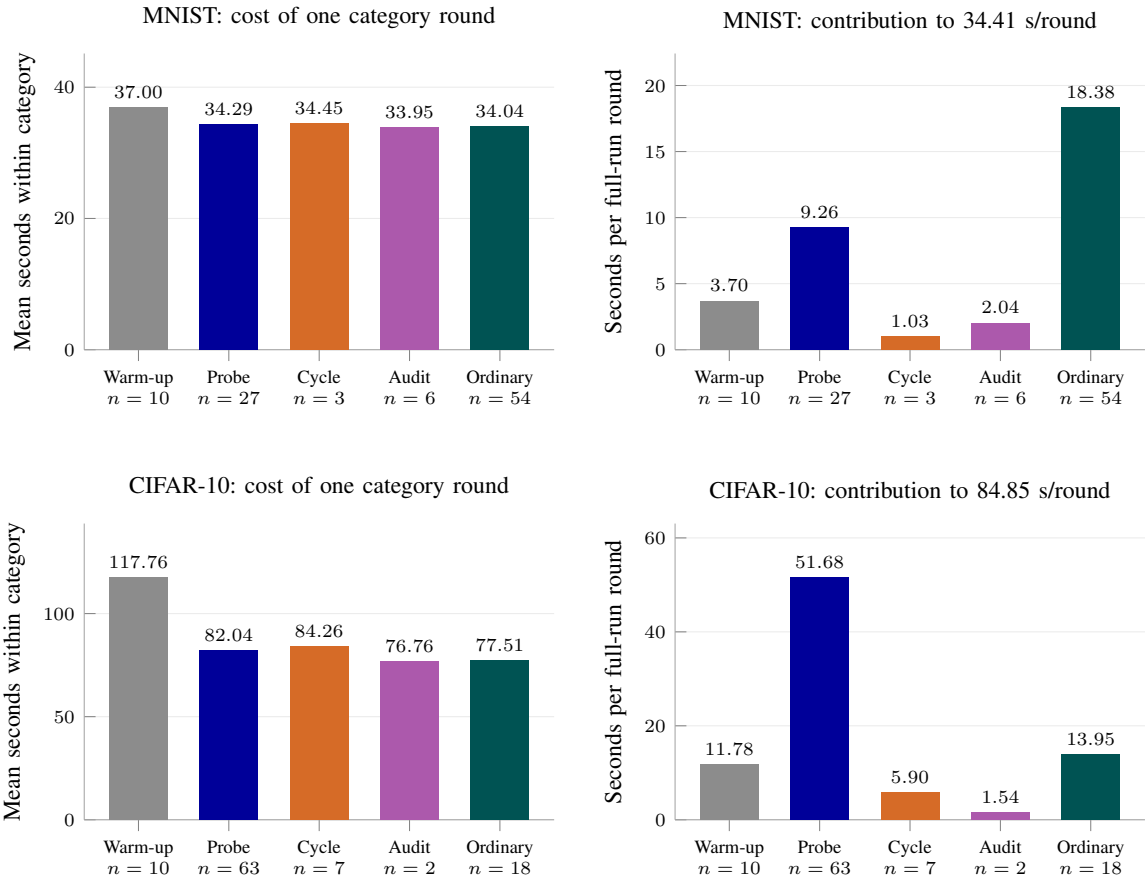


Fig. 8. Where our 100-round runtime is spent (non-IID FR3, seed 42, 40% free riders). Rows separate MNIST and CIFAR-10. Left: mean duration of a complete round in each category. Right: summed duration for that category divided by 100; the five bars add to the full-run mean printed in the title. Labels give category counts n. Probe excludes cycle-decision rounds; Cycle includes the final cycle decision; Audit is a single audit round. Categories are mutually exclusive. A cycle bar includes client training, aggregation and evaluation: it is NOT detector-only time. Axes scale independently; all bars start at zero.

TABLE V

MEASURED STD-DAGMM COMPONENT TIMES (SECONDS PER ROUND, MEAN ± SAMPLE SD, 100 ROUNDS PER DATASET).

Component	MNIST	CIFAR-10
Client training	33.986 ± 1.539	76.208 ± 4.164
Simulated FR response	0.062 ± 0.006	0.255 ± 0.055
Server aggregation	0.042 ± 0.004	0.189 ± 0.043
Global evaluation	2.144 ± 0.073	4.547 ± 0.227
Update preparation	0.045 ± 0.004	0.127 ± 0.015
Detector fitting	13.629 ± 0.467	45.029 ± 1.671
Detector scoring	0.278 ± 0.011	0.856 ± 0.035
Flag decision	0.000 ± 0.000	0.027 ± 0.024
All detector work	13.952 ± 0.476	46.040 ± 1.695
Combined compute	50.186 ± 1.943	127.239 ± 5.478

Our historical training, aggregation, detection and trap-generation components: not separately recorded. Network transmission: not measured in either local simulator. STD has no traps. All detector work includes preparation, fitting, scoring and decisions; it overlaps those table rows and must not be added a second time.

TABLE VI

NATIVE-WORKFLOW TIMING OVER 100 EXECUTED ROUNDS PER METHOD AND DATASET. ALL ENTRIES ARE MEASURED; THIS IS NOT A CONTROLLED SPEEDUP EXPERIMENT.

Dataset	Workflow / horizon	Rounds	Mean s/round	Sum (min)
MNIST	Ours, measured full horizon	100	34.41	57.35
MNIST	STD-DAGMM, measured full horizon	100	50.19	83.64
CIFAR-10	Ours, measured full horizon	100	84.85	141.41
CIFAR-10	STD-DAGMM, measured full horizon	100	127.24	212.06

TABLE VII

OUR 100 MEASURED ROUNDS GROUPED BY EVENTS. WITHIN-CATEGORY MEAN IS NOT DETECTOR-ONLY OVERHEAD; AMORTIZED CONTRIBUTION IS CATEGORY SUM / 100.

Dataset	Round category	Count	Within-category mean (s)	Contribution (s/round)
MNIST	Warm-up	10	37.00	3.70
MNIST	Probe without cycle decision	27	34.29	9.26
MNIST	Full-cycle decision	3	34.45	1.03
MNIST	Single audit	6	33.95	2.04
MNIST	Ordinary	54	34.04	18.38
CIFAR-10	Warm-up	10	117.76	11.78
CIFAR-10	Probe without cycle decision	63	82.04	51.68
CIFAR-10	Full-cycle decision	7	84.26	5.90
CIFAR-10	Single audit	2	76.76	1.54
CIFAR-10	Ordinary	18	77.51	13.95

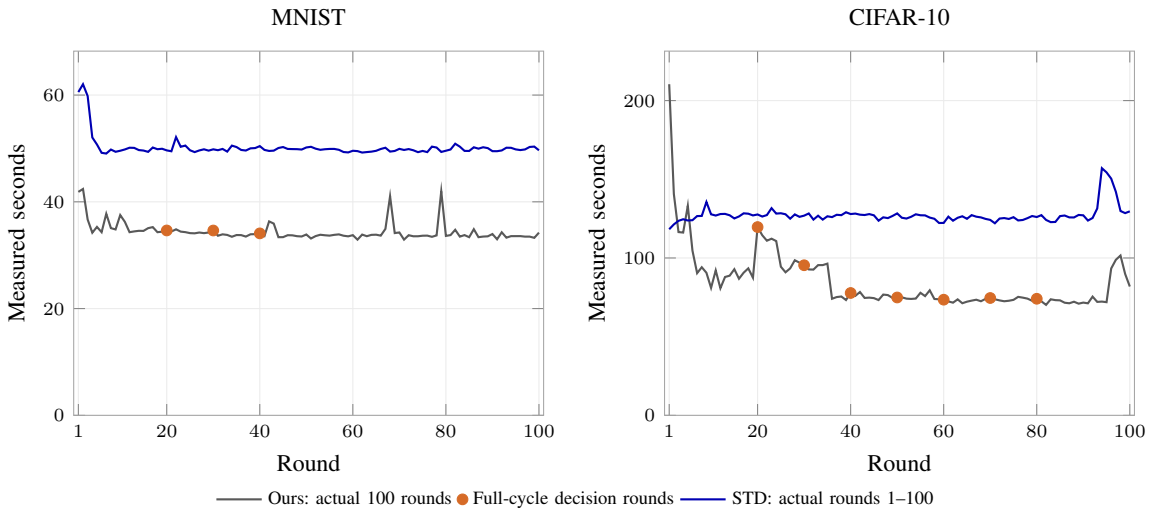


Fig. 9. Actual per-round durations over 100 executed rounds for both methods. Our trajectories include changing active-client counts and probing schedules; STD-DAGMM retains all 100 clients. Markers indicate full-cycle decisions, not their isolated computational cost. Axis scales differ by dataset.

VI. Result Analysis

A. Effect of Attack Behavior

FR1 is consistently removed at the earliest allowed round. Exact sendback provides a distinct evidence path, so its success does not validate the statistical separation gate. Near-converged honest zero updates would still require separate examination in other training regimes.

FR2 is removed by round 30 in all settings despite avoiding equality. The observed responses are sufficiently distinct under the configured random update distribution and repeated traps. This is evidence about the tested amplitude, not proof that arbitrary fabricated noise will be detected.

FR3 is the critical heterogeneous-data case. Its current-inclusive history can yield responses that overlap honest update geometry. In non-IID CIFAR-10 at 40% in the representative replication, all 40 free riders receive full norm certificates at rounds 20 and 60; at 30%, all 30 receive them at rounds 40 and 80. Ambiguous intervening cycles do not increment these counts. Retention permits the later fresh certificate to confirm removal. This recorded sequence explains how the memory rule operates; it is not a matched ablation proving that memory alone caused the outcome under a counterfactual training trajectory.

FR4 is removed by round 30 in the representative replication, while additional replications require up to round 90. Its received-trajectory estimate includes traps, which plausibly contributes to separation from genuine optimization. This is an interpretation of the mechanism, not a measurement of the attacker's inference ability. A defense-aware attacker could treat repeated or unusual received states differently and needs a separate evaluation.

B. Dataset, Partition, and Attacker Share

MNIST FR1 to FR3 complete free-rider removal at round 30, while some MNIST FR4 and non-IID CIFAR-10 executions require additional evidence cycles. This shows that neither attack identity alone nor final recall fully describes difficulty. The interaction between honest-data heterogeneity, model state, and fabricated response geometry matters.

In the representative replication, changing the share from 40% to 30% preserves complete detection but delays the difficult CIFAR-10 FR3 case from round 60 to round 80. A lower attacker share therefore does not imply faster discovery. Cohort support and the distribution of the surrounding honest population both change; the server does not use the known simulation share to force a quota or boundary.

The 30% runs also train 70 honest clients rather than 60. This increases genuine local work and helps explain longer simulator round times without implying a more expensive statistical rule. Non-IID client dataset sizes differ, so the increase need not equal the 16.7% change in honest-client count. The slightly higher achieved accuracies at 30% are consistent with more honest training, but no causal effect is established without matched controls.

C. Screening Quality Versus Removal Safety

The evaluated procedure achieves high removal precision, but the additional replications show that repeated evidence does not eliminate false positives. The central unresolved issue is that some non-IID honest populations remain almost entirely candidate-labelled after attackers disappear. This can arise when relative statistical comparisons continue to find weak differences among honest clients. Such labels do not, by themselves, exclude ordinary updates, but they limit the meaning of calling R "safe" or C "malicious." We interpret R as unflagged, C as unresolved, and S as freshly certified suspicion.

Temporary exclusion also remains a real cost. Most honest exclusions are deliberate trap responses, not quarantine. In ordinary successful MNIST runs, 660 honest client-rounds are withheld at 40% and 770 at 30%, corresponding to eleven trap rounds per honest client over the 100-round horizon. The exceptional FR3 follow-up adds further probing. A deployment should measure whether this forgone aggregation delays reaching a target utility, rather than reporting permanent-removal precision alone.

D. What the Current Evidence Does Not Establish

The repeated experiments assess variation across the tested seeds. Wider validation should include honest-only populations, longer training horizons, matched ablations, and baseline comparisons under identical trajectories. Client observations within a run are correlated, and complete recall in these experiments does not establish universal detection. Variations of attacker history length, noise amplitude, trap strength, local epoch count, non-IID concentration, and model architecture remain necessary robustness tests. In particular, the influence of CIFAR BatchNorm buffers should be separated from trainable-weight evidence.

VII. Complexity and Lightweight Design

A. Operation Counts and Scaling

Let N be active clients, P floating model coordinates (including floating buffers), L floating tensors, T rounds, B bootstrap replicates and $K = \lceil \sqrt{N} \rceil$ histogram bins. The following are implementation bounds, not measured wall-clock predictions. We count model scanning separately from local optimization and ordinary aggregation.

The leave-one-out implementation rebuilds $N-1$ peer profiles for every client and trap. Profile medians and distances require $O(N^2 L)$ coordinate work; repeated scalar medians in Python's `statistics.median` add $O(N^2 \log N)$ sorting work. Thus, calling the whole method linear in client count would be incorrect. Histogram binning and bootstrap resampling operate only on scalar scores, not on P-dimensional model vectors. With M complete cycles, the detector work is bounded by

$$W = O(TNP) + O\big(M\left[N^2(L+\log N) + B(N+K)\right]\big). \tag{16}$$

Single-trap audits add corresponding one-view scoring passes; they cannot authorize permanent removal. Dense monitoring supplies more observations but only one preselected A/B pair per client enters each complete-cycle decision.

At $N = 100$, the two-trap profile comparison visits $2N(N-1)L = 158{,}400$ profile coordinates for MNIST and $435{,}600$ for CIFAR-10, before constants for medians and distances. One bootstrap view draws $BN = 100{,}000$ scalar indices; three views process three such sets using reused draws in the current implementation. These counts explain why the evidence layer is compact relative to repeated full-model training, but they do not measure its elapsed time.

B. Payload Sizes, Memory Boundaries and Communication

Table IX derives concrete payload sizes from the implemented architectures. The compact float32 pair payload is $2N(L+3)\cdot 4$ bytes under the stated packed representation; the three scalar slots are an illustrative norm/round/flag allowance, not the actual Python object layout. The two trap models occupy $2P \cdot 4$ floating bytes. Actual dictionaries, lists, Python scalars, allocator overhead, fingerprints and bootstrap temporaries increase memory. For example, bootstrap index gathers and bin-index arrays coexist with resample/count arrays; their two listed sizes must not be relabelled as peak bootstrap RSS.

The decision working state is $O(P + NL + BN + BK)$, plus $O(N)$ counters and fingerprints. In contrast, the research logger accumulates per-round/probe rows, including tensor profiles, so the full simulator's retained evidence can grow as $O(TNL)$ in dense probing. Ordinary aggregation also retains model responses, and the simulator stores attack histories. The measured process RSS in Table III includes these costs; it is not evidence that the entire implementation uses only compact decision-state memory. Streaming logs and aggregation would be deployment optimizations, not features benchmarked here.

No extra local optimization pass or auxiliary classifier is required. Under the existing full-participation request/response schedule, traps replace ordinary models rather than adding a second exchange; challenge responses are discarded from aggregation. The nominal model-payload order remains $O(NP)$ per round, excluding transport/serialization overhead. This does not make communication "free": useful contributions are withheld, so reaching a target accuracy can require more rounds. Network-byte measurements and controlled detector-only profiling are still necessary. We use *lightweight* to mean compact features, no auxiliary learning, and constant-sized per-client evidence memory, not an unmeasured universal speed or energy advantage.

VIII. Security Analysis

A. Decision Inputs and Attack-Label Separation

The detector receives an allowlist of client/trap IDs, raw norms, profile distances, robust scores, and exact-sendback indicators. The simulator's attacker labels and behavior choices are not passed to screening, geometric tests, or the client state machine. Evaluation prevalence determines how many attackers the simulator creates, not how many removals the detector attempts to achieve. This code boundary reduces direct evaluation leakage; it does not make the development process independent of previously observed results.

B. Confirmation Invariants

Three operational safeguards follow from the state-transition rules. First, statistical removal requires the same component on both selected traps and a second distinct full cycle: one trap, mixed norm/profile evidence, or an ambiguous result cannot independently supply the required confirmation. Second, previously stored evidence does not authorize a new removal without a fresh current certificate. Third, stored counts and weak C evidence cannot renew quarantine; unconfirmed review expires unless fresh strong evidence returns. These are protocol invariants, not probabilistic security bounds.

The guarantees are intentionally narrower than "no false positives." An honest client can satisfy the same geometric gate in two cycles. A clean majority and independent trap draws do not make the resulting statistical certificates independent, because clients retain the same data and related model trajectories.

TABLE VIII

INCREMENTAL DETECTOR OPERATIONS AND WORKING STATE. "CYCLE" MEANS A COMPLETE A/B EVIDENCE CYCLE; IT DOES NOT MEAN EVERY TRAINING ROUND. BOUNDS EXCLUDE ORDINARY MODEL AGGREGATION, LOCAL TRAINING, SIMULATOR ATTACK HISTORIES AND PERSISTENT LOGGING.

Component	Work	Working state	Practical interpretation
Norm, profile, exact fingerprint	$O(NP)$ per round	$O(P)$ scratch, $O(N)$ fingerprints	Sequential model reads; no auxiliary training or gradients.
Selected A/B summaries	$O(NL)$ per cycle to store	$O(NL)$	Two compact profiles per active client, not full response histories.
Leave-one-out norm/profile scores	$O(N^2(L + \log N))$ per cycle	$O(NL)$ temporary	Repeated peer-profile medians/distances; Python scalar medians sort.
Three histogram/bootstrap views	$O(B(N + K))$ per cycle	$O(BN + BK)$ scratch	Three views add a constant factor; $B = 1000$.
Geometric gates and memory	$O(N)$ selections/quantiles per cycle	$O(N)$ counters	Fixed component count; no learned detector or history-window reconstruction.
Two model challenges	$O(P)$ generation per cycle	$O(P)$	Two shared trap states, not a distinct trap model per client.

TABLE IX

CALCULATED DETECTOR PAYLOAD SIZES AT $N = 100$, TWO TRAPS, $B = 1000$: ARCHITECTURE-DERIVED ESTIMATES, NOT MEASURED HEAP USAGE.

Model	Parameters	P	L	Two traps (MiB)	Pair payload (KiB)	Index array (MiB)	Counts (MiB)
MNIST	206922	206922	8	1.579	8.594	0.763	0.076
CIFAR-10	620810	621258	22	4.740	19.531	0.763	0.076

P includes floating BatchNorm buffers; L counts floating tensors. Pair payload assumes packed float32 profiles plus three scalar slots per trap/client. Bootstrap columns show two principal arrays, not peak scratch or an upper bound. No detector-only RSS was recorded.

C. Evasion, Identifiability, and Trap Secrecy

Consider two executions with identical server-observable response histories, public identities, and server randomness, one produced by honest training and the other by fabrication. A detector whose transition function depends only on those observations has the same state trajectory in both executions. Therefore, update-only screening cannot prove genuine computation against every attacker. The proposed method must be evaluated by how well its challenges separate specified behaviors, not as a cryptographic proof of training.

The evaluated attackers are not given trap labels, but several real-world signals could reveal auditing: repeated frozen states, abrupt distributional changes, or communication between clients receiving different models. Moreover, the simulator derives reproducible randomness from configured seeds; it does not establish unpredictability against an adversary that knows those seeds. Deployment would require server-secret randomness and an evaluation of defense-aware response strategies. Even secret randomness alone would not establish indistinguishability of traps from ordinary models.

D. Honest-Client Safety and Long-Term Accumulation

There is no fitted numerical z-score cutoff for statistical removal, but the policy is not parameter-free. Trap scale, histogram binning, bootstrap confidence, cohort support, confirmation count, and review/clearance lengths are design choices. A stable boundary is not a calibrated false-removal probability. At 100 clients the minimum statistical cohort support is ten; an isolated nonzero free rider can therefore escape this cohort gate.

Certificate memory deliberately has no fixed age limit. If an honest client's atypical geometry generates repeated strong evidence without two complete R cycles, it can eventually be removed. The profile gate's concentration alternative and persistent candidate populations make this a material long-horizon risk. The honest removals observed in the repeated experiments show that false evidence can lead to exclusion within 100 rounds. Longer honest-only and heterogeneous-population experiments are needed before making deployment safety claims.

E. Privacy, Collusion, and Availability

The protocol is not directly compatible with standard secure aggregation and does not provide differential privacy [11]. Avoiding client-reported loss and raw data access reduces required inputs but is not a privacy proof: identifiable returned models may still leak information. Colluding clients could share a genuinely trained response or infer the trap schedule, and Sybil identities could distort the assumed honest-majority reference. Neither attack is evaluated.

All active clients and complete A/B coverage are required for a certifying cycle. Missing/nonfinite measurements cannot certify, and the current simulator aborts on a nonfinite client response rather than implementing a complete availability defense. Dropout tolerance, malicious-message handling, restart integrity, authentication, and denial-of-service resistance require additional engineering and experiments. Checkpoint recovery preserves computation and evidence state after interruption; it does not create new evidence of honesty.

F. Computational and Communication Exposure

The bounds in Section VII expose quadratic-in-population leave-one-out work and transient bootstrap memory. A large population or model can make auditing expensive even without

learned detector training. Deliberately withheld trap responses and repeated honest follow-up are an availability and utility cost, not merely an implementation detail. Communication bytes, time to target utility, energy, and isolated detector work remain unmeasured.

IX. Conclusion

This study evaluates a lightweight, server-observable challenge approach to free-rider screening, with strict norm separation, component-specific evidence memory, and bounded review. Evaluation across multiple random seeds on MNIST and CIFAR-10 achieves 100.00% recall and pooled precision of 99.91%. Most executions complete detection by round 30, with more difficult cases requiring up to round 90. The three honest removals show that repeated confirmation does not eliminate false positives. The results show useful separation under the evaluated training and attack configurations, while the candidate and quarantine results show why final recall and precision alone are insufficient.

Further validation should include broader client populations, honest-only training, and tests of adaptive/mixed attacks and longer horizons. Matched ablations and detector-only profiling should distinguish the contributions of traps, geometric gates, and memory from ordinary training and simulator costs. The method is a promising empirical screening mechanism, not a universal verifier of local computation.

Appendix A
Representative Per-Run Results

Tables X and XI retain detailed outcomes from the representative replication. Full records for the repeated experiments, including the honest removals, are provided in `generated/multiseed/per_run.csv`. Group means and accuracy standard deviations are retained in `generated/multiseed/table1_means.csv`.

Appendix B
Supporting Trajectories and Score Geometry

Figure 10 retains the representative cumulative detection trajectories, including the delayed non-IID CIFAR-10 FR3 cases. Figure 11 restores the full training horizon and shared 0–1 scale for interpreting the main-text accuracy zooms. Figures 12 and 13 show the complementary 30% client clouds using the same paired-score definition and monotone display transformation.

Appendix C
Reproducibility and Interpretation Notes

Table I is generated by `build_multiseed_table.ps1`. It validates completion markers, configuration coverage, training rounds, initial A/B coverage, removal identities and timing, summary counts, final accuracy, detector policy, and six recorded implementation hashes across all replications. Seed identifiers and input hashes are retained in `generated/multiseed/provenance.json`, with individual outcomes and grouped means in the accompanying CSV files. These records include failed detections and honest removals rather than selecting only favorable outcomes.

The representative diagnostic assets are retained from the earlier development replication. Their input hashes are recorded in `generated/provenance.json`, and the six matching source files are copied to `evidence/source/`. These hashes describe the recorded implementation and do not attest the complete historical environment. The score exports retain raw paired coordinates and per-class means, medians, quartiles, finite counts and undefined counts for every complete evidence cycle. Additional resource-comparison inputs and the architecture hash are recorded in `generated/visual_provenance.json`. The historical v15.3 cost table is separate context and is never pooled with v15.4 detection totals.

The diagnostic tables and figures are preserved from v6, with captions updated to distinguish individual executions from the averages in Table I. Compile `paper_v7.tex` with PDFLaTeX and BibTeX, or use the accompanying `build.ps1`. Previous paper versions and all experiment logs remain unchanged. The older v2/v8 results are not pooled with the current method: in particular, FR1 semantics, confirmation, reference construction, and screening policy differ.

Interrupted runs resume their last published checkpoint with loader/RNG state, attack histories, selected probe positions, quarantine and confirmation memory. The runtime table reports logged execution measures rather than powered-off calendar waiting. None of the reported resource values should be relabelled as detector-only overhead or interpreted as evidence of privacy/security guarantees.

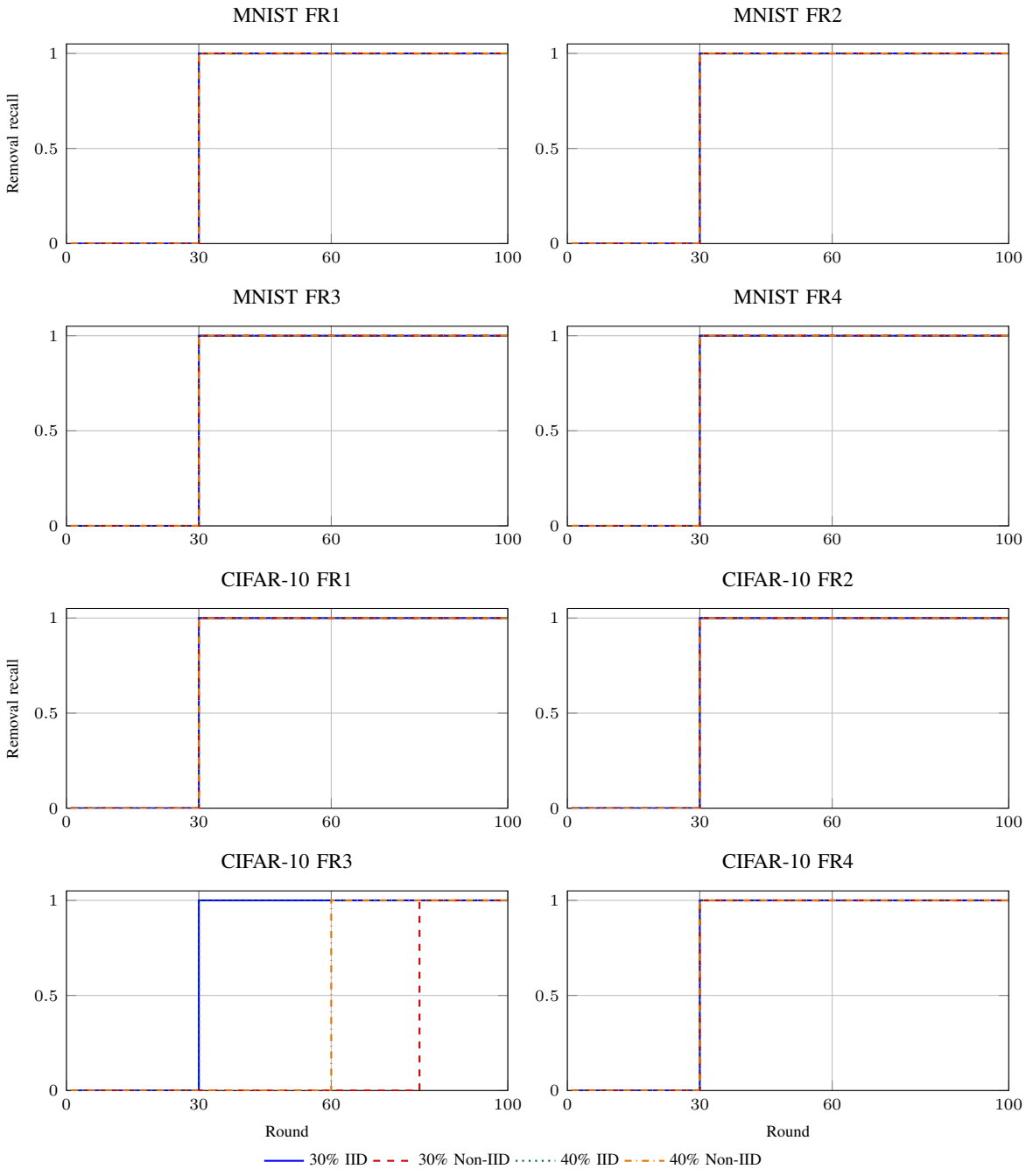


Fig. 10. Cumulative free-rider removal recall in the representative replication. Enlarged two-column panels identify dataset and attack in each title; the four lines distinguish partition and attacker share. Coincident curves overlap. Complete outcomes are retained, including incomplete detection if present.

TABLE X
DETECTION AND UTILITY OUTCOMES FOR THE REPRESENTATIVE MNIST REPLICATION.

FR (%)	Partition	Attack	TP	FP	FN	t_{all}	Accuracy (%)	C_H	Q_H	q_H
30	IID	FR1	30	0	0	30	99.02	1	0	0
30	IID	FR2	30	0	0	30	99.00	5	0	0
30	IID	FR3	30	0	0	30	99.00	8	0	0
30	IID	FR4	30	0	0	30	98.82	2	0	0
30	Non-IID	FR1	30	0	0	30	99.01	7	0	0
30	Non-IID	FR2	30	0	0	30	99.00	6	0	0
30	Non-IID	FR3	30	0	0	30	98.98	8	0	0
30	Non-IID	FR4	30	0	0	30	98.96	6	0	0
40	IID	FR1	40	0	0	30	98.90	1	0	0
40	IID	FR2	40	0	0	30	98.91	5	0	0
40	IID	FR3	40	0	0	30	98.85	5	0	0
40	IID	FR4	40	0	0	30	98.77	5	0	0
40	Non-IID	FR1	40	0	0	30	98.96	2	0	0
40	Non-IID	FR2	40	0	0	30	98.96	7	0	0
40	Non-IID	FR3	40	0	0	30	98.93	34	1	20
40	Non-IID	FR4	40	0	0	30	98.98	12	0	0

TABLE XI
DETECTION AND UTILITY OUTCOMES FOR THE REPRESENTATIVE CIFAR-10 REPLICATION.

FR (%)	Partition	Attack	TP	FP	FN	t_{all}	Accuracy (%)	C_H	Q_H	q_H
30	IID	FR1	30	0	0	30	78.07	2	0	0
30	IID	FR2	30	0	0	30	77.93	0	0	0
30	IID	FR3	30	0	0	30	78.17	2	0	0
30	IID	FR4	30	0	0	30	78.03	7	0	0
30	Non-IID	FR1	30	0	0	30	75.80	8	0	0
30	Non-IID	FR2	30	0	0	30	75.58	2	0	0
30	Non-IID	FR3	30	0	0	80	75.36	70	1	20
30	Non-IID	FR4	30	0	0	30	75.78	7	0	0
40	IID	FR1	40	0	0	30	77.46	0	0	0
40	IID	FR2	40	0	0	30	77.65	0	0	0
40	IID	FR3	40	0	0	30	77.27	4	0	0
40	IID	FR4	40	0	0	30	77.36	8	0	0
40	Non-IID	FR1	40	0	0	30	75.28	9	0	0
40	Non-IID	FR2	40	0	0	30	75.43	4	0	0
40	Non-IID	FR3	40	0	0	60	74.89	52	9	180
40	Non-IID	FR4	40	0	0	30	75.14	7	0	0

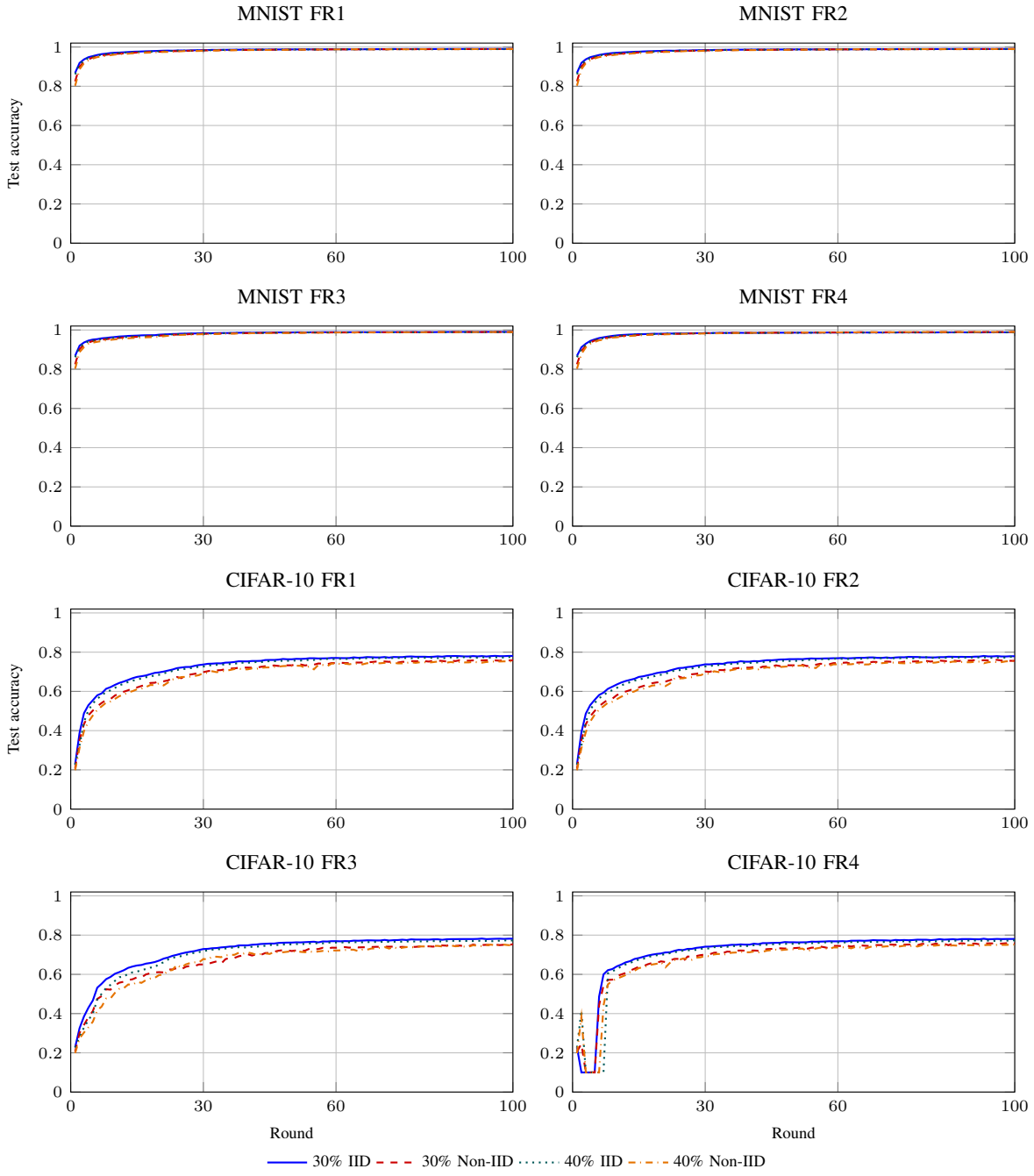


Fig. 11. Full, unzoomed global test-accuracy trajectories for the representative replication, providing context for the late-training detail in the main text.

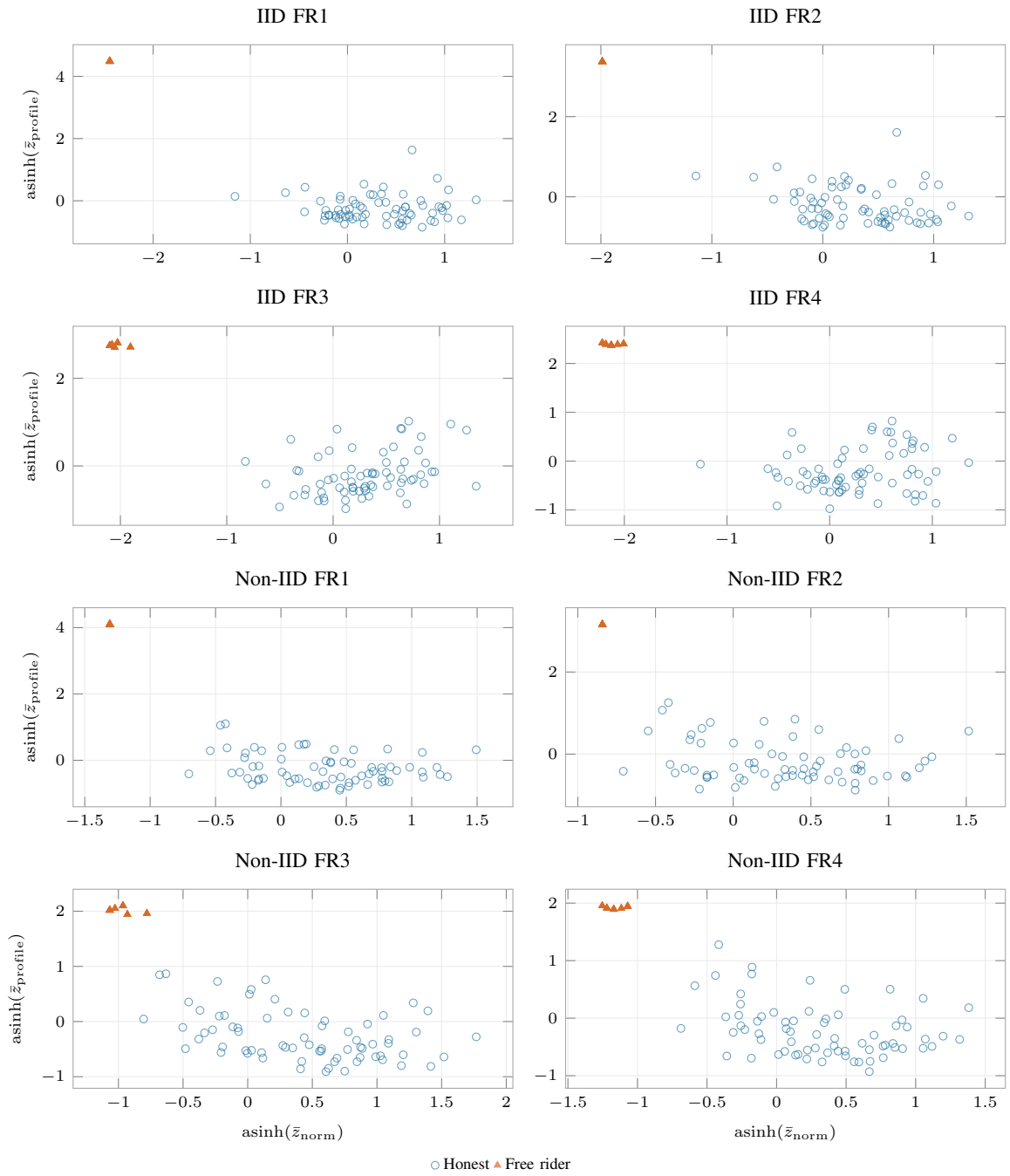


Fig. 12. MNIST: round-20 score geometry at 30% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

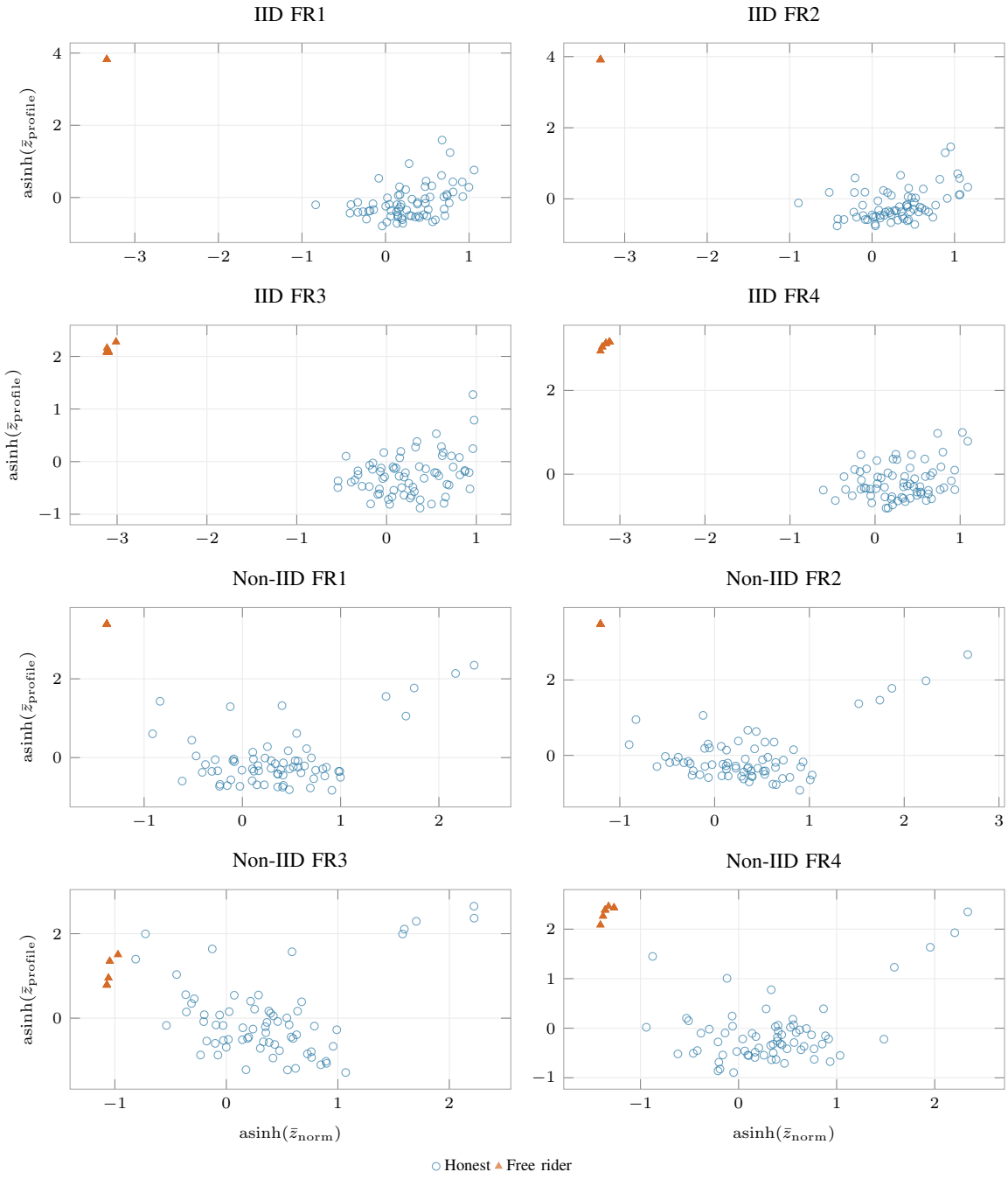


Fig. 13. CIFAR-10: round-20 score geometry at 30% free riders (all eight settings). One point is one client, averaging the A/B norm-z and profile-z scores separately. The monotone asinh display compresses extreme tails without clipping; zero remains zero, but distances are transformed. Panels autoscale independently. Labels color evaluation only, never references or decisions. 0 points have undefined coordinates and are omitted; coincident points overlap. No plotted line is a removal cutoff.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[2] N. Rieke, J. Hancox, W. Li *et al.*, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, 2020, art. no. 119. [Online]. Available: https://www.nature.com/articles/s41746-020-00323-1

[3] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. [Online]. Available: https://arxiv.org/abs/1912.04977

[4] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 108. PMLR, 2020, pp. 2938–2948. [Online]. Available: https://proceedings.mlr.press/v108/bagdasaryan20a.html

[5] J. Lin, M. Du, and J. Liu, "Free-riders in federated learning: Attacks and defenses," arXiv preprint arXiv:1911.12560, 2019. [Online]. Available: https://arxiv.org/abs/1911.12560

[6] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 130. PMLR, 2021, pp. 1846–1854. [Online]. Available: https://proceedings.mlr.press/v130/fraboni21a.html

[7] J. Wang, X. Chang, J. Mišić, V. B. Mišić, and Y. Wang, "PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2024. [Online]. Available: https://arxiv.org/abs/2207.07292

[8] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, "FRIDA: Free-rider detection using privacy attacks," *Journal of Information Security and Applications*, vol. 97, 2026, art. no. 104357. [Online]. Available: https://arxiv.org/abs/2410.05020

[9] Z. Zhu, J. Shu, X. Zou, and X. Jia, "Advanced free-rider attacks in federated learning," in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021. [Online]. Available: https://neurips2021workshopfl.github.io/NFFL-2021/papers/2021/Zhu2021.pdf

[10] B. Wang, H. Li, X. Liu, and Y. Guo, "FRAD: Free-rider attacks detection mechanism for federated learning in AIoT," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. [Online]. Available: https://doi.org/10.1109/JIOT.2023.3298606

[11] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. [Online]. Available: https://research.google/pubs/practical-secure-aggregation-for-privacy-preserving-machine-learning/

[12] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. [Online]. Available: https://doi.org/10.1109/TSMC.1979.4310076

[13] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: https://doi.org/10.1214/aos/1176344552

[14] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. [Online]. Available: https://doi.org/10.1109/5.726791

[15] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/cifar.html